

STAMM: Soft sensor monitoring and maintenance framework for Machine learning Models

Abstract

Artificial intelligence (AI) and machine learning (ML) models as well as Digital shadows and Digital twins as well as Digital shadows and Digital twins have been, and are being, developed for a variety of applications, including industrial process supervision, bioprocess optimization, energy management, and environmental monitoring. In cases where models have the aim to classify, predict, or forecast a variable of interest then these models must become deployable in real-time so that their predictions can be used in decision making and any corrective actions can be taken to steer the system back on track in a timely fashion. However, once deployed these models must also undergo routine maintenance to ensure that they continue to perform adequately for the task at hand. Here, we present the Soft sensor monitoring and maintenance framework for Machine learning Models (STAMM), a framework designed to support supervision, deployment, and upkeep of soft sensor models in industrial environments. STAMM integrates six key components: (1) data acquisition from sensors and actuators; (2) a time-series database; (3) a workflow management system; (4) a model repository; (5) real-time monitoring dashboards; and, (6) drift detection modules to facilitate the reliable execution and oversight of ML-based soft sensors. STAMM has been built and validated within a biotechnology project, using an industrial-scale fed-batch fermentation as the primary guiding use case. STAMM however, remains fully applicable to a broad range of industrial process scenarios.

Keywords:

- Machine learning operations (MLOps)
- Soft-sensors
- Machine learning (ML) and artificial intelligence (AI)
- Digital Twins and Digital Shadows
- Industry4.0 (I4.0)

1. Introduction

1.1 Industry 4.0

The next phase of industrial evolution (in manufacturing) is referred to as Industry 4.0 (herein I4.0) and will be driven by progress in digital technologies (Soori et al., 2026); cyber-physical systems; big data; cloud computing; and, artificial intelligence (AI) and machine learning (ML). It is envisioned that in this, fourth iteration, industrial processes will involve

arrays of sensors that will monitor, derive information about, and track physical processes, thereby generating data that can be used to control, direct, or operate a process, or system, under observation. Such data will be essential for (process) monitoring (*i.e.*, observation, surveillance); fault prediction, detection, and diagnosis; (product) testing; and, (product) quality assessment (Rahman et al., 2023). At the same time this data is also required for the development of models that may emulate or simulate the behaviour of physical processes. The purpose of such simulations will be to enhance productivity, support optimisation, and make better, more up-to-date, data-driven, (commercial) decisions. Once sensor values have been obtained and/or models have produced output and a decision has been reached then control systems will drive a process or system via a feedback to induce a controller and/or actuator (*e.g.*, stepper motors, solenoids, DC motors, hydraulic rams, pumps, heaters, coolers) to perform a corrective action (Kant & Gurung, 2023).

Ultimately large-scale data collection from such systems via sensors and actuators, including the response of a system under various conditions and control strategies, will allow for the creation of models of 'behavior'. I4.0 will therefore, enable the twinning of the physical process in the digital world (Santos & Sant'Anna, 2024) referred to as Digital Shadows and Digital Twins (Appl et al., 2021; Gargalo et al., 2021; Margaria & Schieweck, 2019). These differ in how they communicate: Digital shadows are virtual representations that track real-world phenomena by a one-way flow of communication (Kritzinger et al., 2018). Whereas, Digital twins are real-world phenomena that are linked by a two-way flow of communication with a virtual representation (Bergs et al., 2021). Both of which differ from 'traditional' models by (i) having a relationship to a real-world, physical, phenomenon or object that (ii) has a dynamic, 'evolving', set of data regarding that phenomenon which (iii) updates or adjusts the state variables (Wright & Davidson, 2020). The virtual representations of real-world phenomena provided by either a Digital Shadow or Digital Twin would enable monitoring, simulation, analysis, and testing of control scenarios without adversely affecting the real-world entity facilitating enhanced decision making, *e.g.*, aiding optimization of process; development of control strategies; and forecasting and scheduling maintenance.

However, at present many systems lack sensors that can adequately and reliably convey the value of variables of interest in a timely fashion - in or near to 'real-time'. With measurements of key variables occurring only after any possible chance that the values obtained could affect the outcome of any decision-making. For many potential Digital Twins, especially those in the life sciences, the lack of real-time data for variables of interest remains a bottleneck to their development (Knibbe et al., 2022). A bottleneck that must be overcome if Digital Twins are to be usable in the future.

1.2 Soft sensors

1.2.1 Types of Soft Sensor

In situations where certain, key, variables are not directly measurable by a sensor in real-time, such as offline measurements that require a form of gas chromatography or mass spectrometry (Rosenthal et al., 2024), then a substitute, or proxy, for such variables must be

found to allow for real-time decision making (Knibbe et al., 2022). One potential approach is to utilise a proxy for such variables, converting the available real-time measurements into real-time estimations of these variables via software based sensors or 'soft sensors' (Kadlec et al., 2009); 'soft sensors' being a combination of "software" given that the models are usually computer programs, and "sensors" since the models provide similar information as physical sensors. Soft sensors use real-time data collected by physical sensors and actuators to estimate parameters that would otherwise be difficult, expensive, or impossible to measure directly (**Figure 1**). Of course, this approach assumes that measurable variables contain some information that facilitates the prediction of the variable of interest. Although this is dependent upon the soft sensor modeling approach which can be classified into those that are (i) model-based; (ii) data-driven (Fortuna et al., 2007); or, (iii) a hybrid approach of the two. Model-based soft sensors are based on the first principle of physical or chemical laws to provide detailed process description whereas, data-driven soft sensors are created based on large datasets generally using ML techniques(Yan et al., 2017).

1.2.2 Soft Sensor construction

The process of constructing a soft sensor begins with data understanding and continues through to evaluation (gray boxes in **Figure 2**) in accordance with the 'Cross-industry standard process for data mining' (CRISP-DM) methodology (Wirth & Hipp, 2000). This process typically involves several stages: initially, relevant process variables are identified and collected from legacy data or real-time measurements. Data preprocessing steps such as filtering, synchronization, normalization, and the treatment of missing or noisy data are then applied to ensure data quality and consistency. Subsequently, appropriate input variables (predictors) are selected based on domain knowledge and statistical analysis, followed by model development using either statistical, machine learning, or hybrid modeling techniques. The model is then calibrated and validated using independent datasets to assess its predictive accuracy, robustness, and generalizability under varying operating conditions. Once a satisfactory model is obtained, it can be deployed as a soft sensor to provide real-time estimates of hard-to-measure process variables, with ongoing monitoring and retraining ensuring sustained performance over time.

In industrial processes, ML-based soft sensors have gained increased popularity due to their ability to provide estimations of key process variables and enhance process optimisation and control without the need for additional sensors. As a result, soft sensors have been developed for water and wastewater treatment (Mishra et al., 2025; Shyu et al., 2023; Pattnaik et al., 2021) ; the petrochemical industry (Fernandes et al., 2025; Mojto et al., 2021); the processing of bauxite ore into alumina (aluminum oxide) within alumina refineries (Cregan et al., 2017; Li et al., 2025); as well industrial biotechnology and bioprocessing (Metcalf et al., 2025; Kim et al., 2025; Acosta-Pavas, Corrales, et al., 2024; Acosta-Pavas, Robles-Rodriguez, et al., 2024; Krishna et al., 2021; Zhu et al., 2020).

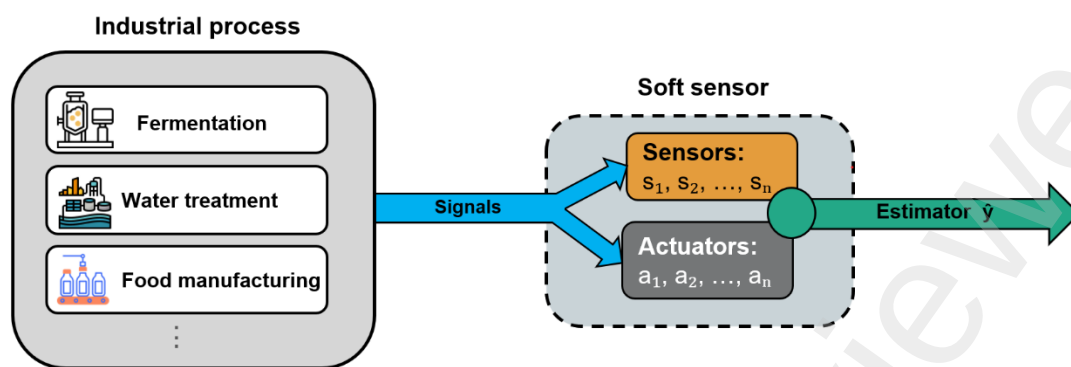


Figure 1. Basic principles of a soft sensor. (Left) Examples of industrial processes. On the right, a graphical representation illustrates the components involved in building a soft sensor, including the measurement signals from sensors and actuators used to generate an estimation.

1.3 Machine learning operations (MLOps)

As the number of models and simulations that can incorporate real-time data (including ML-based soft sensors, Digital Twins, etc.) and produce an output that can benefit decision making increases, it is important to consider how to deploy them in a production or real-world environment (orange box in **Figure 2**) so that they may steer processes in real-time. It is here where the application of Machine Learning Operations (MLOps) gains importance (Metcalf et al., 2025; Liu et al., 2020). MLOps is frequently described as a collection of techniques, tools, practices, or processes for ML deployment (Amrit & Narayanappa, 2025). As it is important to ensure that ML models remain reliable and reproducible in real-world applications. MLOps integrates principles from software engineering (DevOps) with data science workflows to automate and streamline the lifecycle of ML models from data preparation, model training, and validation to deployment, monitoring, and retraining. Of the utmost importance, especially for ML models, is devising ways to address any risks and challenges associated with their use including: performance degradation due to data drift or shifts between training and production environments; short and long term dependency issues relating to software and library versions leading to reproducibility issues; and, silent model failures arising from inconsistencies in the underlying computational environment (Treveil et al., 2020). Moreover, as the true performance of an ML model can only be verified under “production” conditions, insufficient monitoring or the absence of robust MLOps infrastructure may result in undetected declines in accuracy and/or stability. Solving these risks and challenges necessitates robust MLOps strategies to govern deployment, monitoring, and continuous improvement.

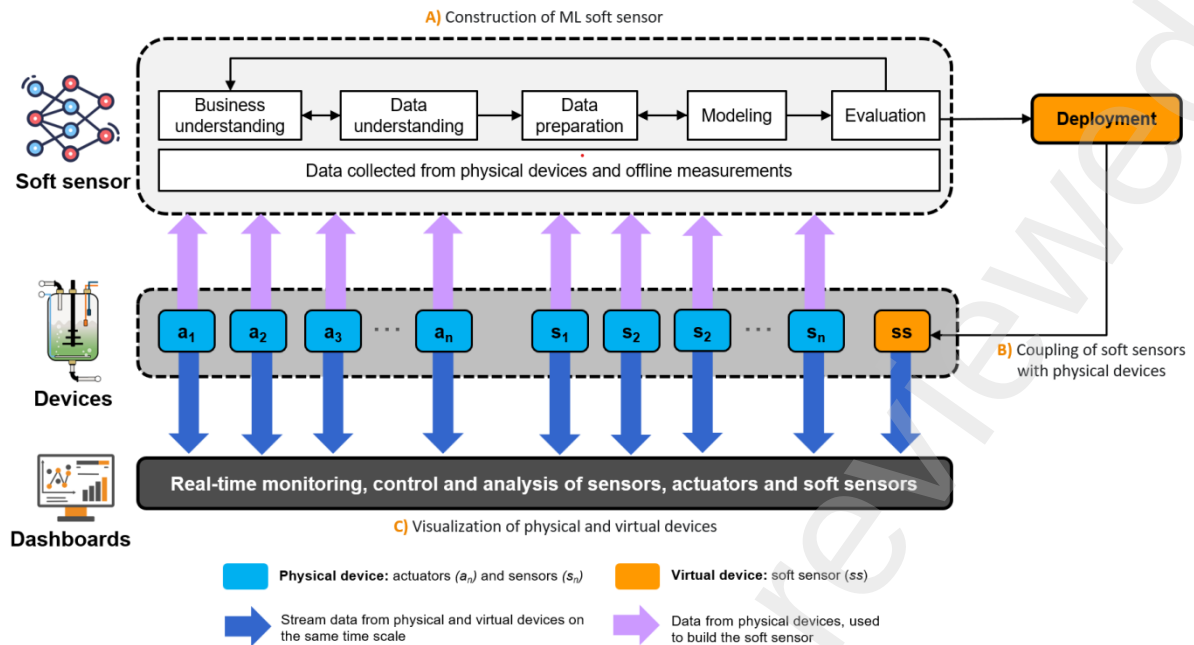


Figure 2. How a soft sensor should operate in real conditions. The first layer corresponds to the construction of the soft sensor, with the phases based on the CRISP-DM methodology (Wirth & Hipp, 2000). The second layer represents the devices that generate measurements, including actuators (a), sensors (s), and estimations provided by soft sensors (ss). Finally, the dashboard layer allows users to visualize all measurements and remains completely transparent to them.

1.4 Aims and objectives

Here the Soft sensor monitoring and maintenance framework for Machine learning Models (STAMM) is presented. STAMM is an open source MLOps tool for soft sensors that enables the deployment of Machine Learning (ML) soft sensors in production. STAMM integrates real-time data from physical sensors and model-based simulations into an interactive dashboard. The quality of the deployed ML soft sensors are monitored for concept drift using a model performance data pipeline. STAMM has been developed as part of a project on Industrial biotechnology - the utilisation of (un)modified organisms to produce the building blocks of, or complete, bio-based products at an industrial scale (Ancelin et al., 2025) - with large-scale penicillin production serving as the example use case (Goldrick et al., 2019, 2015). However, STAMM is sufficiently generic enough to be applied in a wide range of domains and scenarios where MLOps are required (e.g., water and wastewater treatment, the petrochemical industry). This manuscript is organised as follows: **Section 2** presents the STAMM framework in detail, including its overall architecture and its six core components (data acquisition, time-series storage, workflow orchestration, model management, real-time monitoring, and drift detection). **Section 3** provides a discussion of the framework and its relevance for the reliable deployment of ML-based soft sensors, and outlines future developments and perspectives for extending STAMM.

2. Production environment for ML soft sensors

2.1 STAMM Architecture

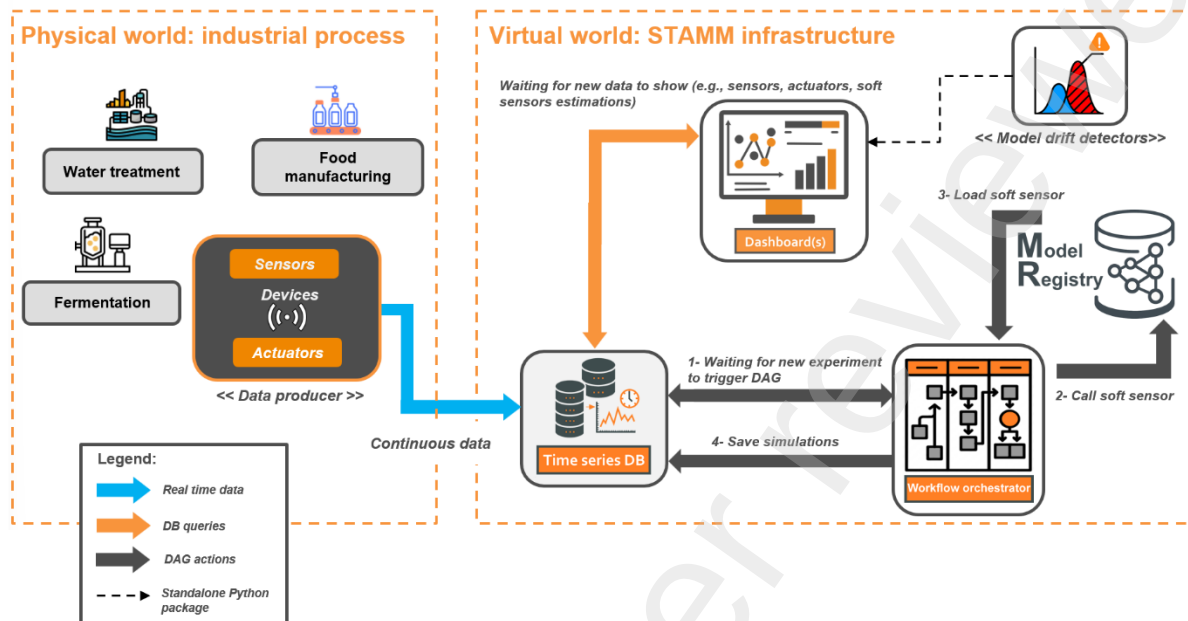


Figure 3. Architecture of the STAMM framework and the interactions between its modules.

STAMM as a framework consists of six components that facilitate short- and long-term execution of soft sensors in real time (**Figure 3**). These include (i) the data producer representing the sensors and actuators of the industrial process; (ii) a time-series database; (iii) a workflow management system; (iv) a (local) repository of ML models where the machine learning soft sensors are stored; (v) the dashboards that enable real-time monitoring of soft sensor simulations; and, (vi) model drift detectors.

Firstly, the data producer – which comprises the sensors and actuators – generates real-time data (**Section 2.2**) which is then stored in the database and subsequently displayed on the monitoring dashboard (see blue arrow in **Figure 3**; **Section 2.5**). Conversely, the "run" pipeline is responsible for executing the machine learning (ML) soft sensors (**Section 2.4**). This is achieved through a Directed Acyclic Graph (DAG) in the workflow management system (**Section 2.3**) to send the simulations to the database. The dashboard then subsequently displays the ML soft sensor simulations (orange arrow in **Figure 3**; **Section 2.5**). Finally, the users play an important key role in the maintenance pipeline, as they are responsible for correcting the simulations and then saving them to the database, deciding whether to retrain the soft sensor if necessary, or, if the soft sensor is experiencing concept drift (see dotted black arrow in **Figure 3**; **Section 2.6**) and cannot be corrected immediately, deactivating it and sending it in for maintenance (**Section 2.6**).

The following sections will provide a detailed explanation of the modules that comprise the STAMM framework, the design requirements and design decisions that underpin them, and the specific functions of each module. Furthermore, the components of the initial prototype are discussed to aid in replication.

2.2 Data producers and time series database

Essential to a MLOps ecosystem is the existence of a reliable and structured data transmission and extraction, transformation, and loading (ETL) pipeline. Fundamentally, this necessitates the presence of one or more data producers capable of continuously supplying measurements, states, and to a programmatically accessible storage layer. In STAMM, this function is realised through the integration of laboratory equipment, middleware for data acquisition, and a time-series database that ensures consistent, and queryable data flows.

2.2.1 Time series database

For the initial development of STAMM, the default setup uses *InfluxDB* (version 2.7.12) as the time-series database. The official docker image (https://hub.docker.com/_/influxdb) provides a reproducible and containerized data infrastructure that automatically provisions organizations, buckets, and scoped access tokens (Ahmad & Ansari, 2017). In InfluxDB, a bucket is a named location where time series data is stored under a defined retention policy and access scope. Within buckets, data is organized using measurements, tags, and fields. A measurement represents a set of related time series data points. Tags are indexed key-value pairs used to store metadata, enabling efficient querying and grouping, such as device identity or project context. Fields contain the actual observed values, such as sensor readings or model predictions.

To organize the data flow three buckets are defined. The first, *stamm_raw*, stores raw sensor signals and process data acquired from various equipment (e.g., temperature, pH, dissolved oxygen). The second, *stamm_predictions*, records model generated outputs (soft sensor estimations) aligned with experimental observations for validation and monitoring. Finally, *stamm_metadata* contains contextual information describing each observed property, including units, display names, and visualization formatting parameters. The division of raw data (*stamm_raw*) and predictions (*stamm_predictions*) into two distinct buckets means that should 'offline' data pertaining to the variable of interest become available in future, or become measured after the deployment, these can be stored separately from predictions.

All buckets share a unified measurement (*device_obs*) and two common tags (*device_id*, *project_name*) to enable consistent cross-bucket queries. Additional specific tags, such as *batch_id* (in *stamm_raw* and *stamm_predictions*), *observed_property* (in *stamm_raw* and *stamm_metadata*), and model related tags (*model_id*, *version*), capture process specific or model specific context (Table 1).

Bucket	Measurement	Common Tags	Specific Tags (examples)	Field (examples)	Description
<i>stamm_raw</i>	device_obs	device_id project_name	batch_id = batch_1 source = sensor observed_property = temperature	value = 300.2	Raw sensor and device data acquired directly from the bioreactor, e.g., temperature, pH, or dissolved oxygen.
<i>stamm_predictions</i>			batch_id = batch_1 source = soft_sensor observed_property = penicillin_concentration model_id = 0009_[Python]_penicillin_LSTM version = v1.2.3	value = 0.085	Outputs generated by machine learning models (soft sensors), aligned in time with experimental observations for validation and monitoring.
<i>stamm_metadata</i>			observed_property = sugar_feed_rate	unit = "L·h ⁻¹ " display_name = "Sugar Feed Rate" decimals = 2	Contextual information describing each observed property, including units, display names, and formatting details for visualization and reporting

Table 1. Example of the unified data structure implemented in InfluxDB for the STAMM system. All datasets share a common measurement name (*device_obs*) and two common tags (*device_id*, *project_name*), ensuring consistent organization and querying across raw sensor data, soft-sensor predictions, and contextual metadata.

2.2.2 Equipment integration layer: LEAF

To transmit data from the equipment to the time-series database, STAMM has the potential to integrate with Laboratory Equipment Adapter Framework - LEAF (<https://leaf.systemsbiology.nl>) as its data acquisition layer (Crowther et al., 2025). LEAF is an open-source, modular tool designed to extract data and metadata from laboratory and industrial devices and to transfer them via MQTT to configurable endpoints, such as InfluxDB instances. Developed in Python (≥ 3.12), LEAF follows an Adapter Architecture that facilitates the monitoring and control of various types of equipment, including bioreactors. Within this architecture, *EquipmentAdapters* serve as the main operational components, integrating other modules (e.g., *ProcessModules* and *PhaseModules*) responsible for event monitoring, data transformation, and output transmission. By coupling LEAF with the InfluxDB-based data model of STAMM, experimental data can be streamed continuously and securely into the MLOps environment, ensuring synchronization between raw measurements, metadata, and model predictions. However, for this proof of concept the data was simulated using Node-Red that automatically injected the data into the timeseries database.

2.3 Workflow management system

2.3.1 Apache Airflow

Deploying, running, and monitoring of a soft sensor requires STAMM to have a workflow management system (WfMS) that can quickly handle scheduling, executing, and monitoring a number of complex tasks at once (e.g., data integration, data processing, data ingestion, ML execution) whilst also tolerating any potential faults (e.g., missing or incorrect data). The WfMS used in STAMM is the open-source Apache Airflow written in the Python programming language having been developed by AirBnB in 2014 (Harensiak & De Ruiter, 2021). Airflow has become widely used for the construction and management of complex data pipelines, including data integration processes such as Extract, Transform, Load (ETL); machine learning workflows; and batch data processing. Airflow allows users to define workflows as Directed Acyclic Graphs (DAG), where each node of the graph represents an individual task and its edges define the execution order and dependencies between tasks (Harensiak & De Ruiter, 2021). Airflow is composed of four components: (i) a scheduler; (ii) the workers; (iii) the web server; and, (iv) the metastore. The scheduler continuously scans DAG definitions to determine whether tasks are due for execution and places them in a queue once their dependencies are met, from which the Workers execute the tasks. To aid the user the Web Server provides a graphical interface that allows for monitoring task execution, viewing logs, and when required manually triggering workflows. Whilst the Metastore is a backend database that stores the workflow metadata such as DAG configurations, task states, and execution history. Because of its modular architecture Airflow enables efficient and fault-tolerant execution of complex workflows that can be scaled up or down depending on the requirements.

2.3.2 Apache Airflow and STAMM

STAMM uses Airflow 3.0.6 (with Python 3.12.11) available as an official docker image (<https://airflow.apache.org/docs/docker-stack/build.html>). In the STAMM architecture, Apache Airflow constitutes the operational engine responsible for coordinating all data acquisition, model invocation, and prediction storage activities within the soft-sensor workflow. While InfluxDB provides the structured time-series backbone for raw measurements, metadata, and model-generated predictions (**Section 2.2.1**), Airflow ensures that these heterogeneous data flows are executed in a temporally coherent and fully automated manner. A dedicated DAG, *STAMM_DAG*, encapsulates this orchestration, enabling real-time operation of STAMM.



Figure 4. Task description for the “STAMM_Predictions” DAG. Bullets indicate the main actions of each task.

The DAG orchestrates four sequential steps (**Figure 4**). First, a health-check is performed that verifies a continued connectivity with the timeseries database. This health check ensures that the system's primary data repository, comprising the *stamm_raw*, *stamm_predictions*, and *stamm_metadata* buckets are available prior to any additional downstream tasks are performed. Next, a data-driven sensor monitors whether the *stamm_raw* bucket has additional or new measurements. Rather than using fixed sampling intervals, the sensor assesses both the temporal 'freshness' and the novelty of incoming measurements within each dynamic tag group (e.g., *device_id*, *project_name*, *batch_id*). It then generates wide-format "snapshots" by pivoting individual *observed_property* fields into unified feature vectors. These snapshots represent the most recent process state and serve as the standardized inputs to all ML models. The third step is model inference wherein, the Model Registry (**Section 2.4**) is called. Rather than embedding models into the platform or requiring manual installation, Airflow communicates with the Model Registry via standardized REST endpoints. For every snapshot passed via Airflow's XComs (Airflow's mechanism for sharing data between tasks), the inference task:

1. *Discovers registered models* for the active project (e.g., via GET `.../{project_id}/list_models/`), retrieving them from a repository capable of storing heterogeneous exported formats (e.g., pkl, joblib, ONNX, h5, rds, pmml, keras, etc.).
2. *Retrieves metadata* for each model when required (e.g., version, architecture, learner type), ensuring traceability and reproducibility of real-time predictions.
3. *Executes inference* through a uniform prediction interface (e.g., POST `.../predict`), regardless of the model's underlying language (Python or R), learner class (interpretable or black-box), or exported format.
4. *Collects versioned predictions*, including explicit model identifiers and harmonized output schemas that are compatible with the STAMM data model.

This design ensures strict interoperability, where models can be invoked programmatically without additional user intervention, satisfying the platform's "low-code" requirement. Finally, the Airflow pipeline stores all predictions in the *stamm_predictions* bucket, preserving the original snapshot timestamp and aligning soft-sensor outputs with their corresponding experimental observations. For each prediction, each entry in the time series database is enriched with the following metadata: *model_id*; *version*; source (*soft_sensor*); and, the predicted *observed_property* that ensures compatibility with cross-bucket queries and dashboard visualizations. Through this integration of database, model registry, and workflow orchestration, Airflow provides STAMM with a reproducible, event-driven, and extensible execution layer. It guarantees that soft-sensor models remain synchronized with data availability, version-controlled through the *Model Registry*, and seamlessly integrated into the unified InfluxDB schema.

2.4 Model Registry

2.4.1 Overview

To be usable as a soft sensor requires the ML soft sensor models to be accessible to STAMM, there are several approaches to overcome this challenge. The first is to include each model as a module with the source code of STAMM; however, this would require that a user would have to re-install, or update, STAMM to the latest version each time the model is updated. A second approach is storing the models as separate Git repositories necessitating that the user install and/or update the model when required. However, both of these solutions necessitate the user perform an such as installing additional packages to match the model's dependencies or even installation of additional programming languages depending upon the model specifics. Ideally, users should experience a 'low code' environment that enhances model interoperability. Therefore, STAMM uses a repository - or a structured collection - of exported models alongside the metadata relevant to their use, that can be accessed programmatically.

ML models can be exported in various formats depending upon the programming language and the packages used in model development. Therefore, the repository had to also facilitate their reuse and integration into different applications and programming languages (e.g., Python, R, etc.). In the case of Python, the repository had to be capable of accommodating models saved in the following formats, including pickle (.pkl), joblib (.joblib), ONNX (.onnx), and HDF5 (.h5) for TensorFlow (Abadi et al., 2016) or Keras (Chollet & others, 2015) based models. Model configurations can be exported as JSON objects. Likewise, the repository must also have to have been capable of handling models developed in the programming language R that are exported as RDS (.rds), RData (.rdata), PMML (.pmml), and serialised objects exported via either the save() or saveRDS() functions. Finally, the repository also had to be constructed in such a way as to permit the incorporation, in time, of additional models, either new or updated versions of existing models. As a proof of concept a standalone web framework was developed using Python's FastAPI package (<https://fastapi.tiangolo.com>) that not only allows for the retrieval and storage of the models and their metadata (e.g., via GET .../{project_id}/metadata/{model_id}) but also allows users to send input data and receive back predictions from the various machine learning models stored within the repository (e.g., via POST .../predict).

2.4.2 ML Metadata

To be usable however, each machine learning model requires a comprehensive set of accessible metadata that comprises essential information that defines and tracks a ML model (Jones, 2019; Walsh et al., 2016). A number of standards and checklists for model and machine learning metadata have been developed (Heil et al., 2021; Walsh et al., 2021; Mitchell et al., 2019) with many of these encompassing information about model construction (e.g., training and test validation); the tools and libraries used during model development, the model architecture; training configuration (e.g., optimiser, loss function, hyperparameters); preprocessing methods used (e.g., feature scaling or temporal aggregation); type of inputs

(e.g., sensor, actuator, precomputed variables, soft sensors); outputs (e.g., lag features or not); and implementation details (e.g., programming languages, dependencies, exported model format). To meet these requirements each model is accompanied by a structured metadata file in YAML format (.yaml) with the following characteristics (the detailed structure of the metadata model is provided in the supplementary material, **Table S1**):

- **Model identification (*model_identification*).** Terms include: *name*; *version*; *UUID*; *author*; *doi*; *creation_date*; and *project* all of which aim to distinguish the model version not only from different versions of the same model but different models.
- **Model description (*model_description*).** Terms include: *learner*; *model_type*; *model_name*; *description*; *language*; *config_files*; *packages*; and *input_time_interval* all of which describe the 'high level' characteristics of the model.
 - The type of model can be broadly described as interpretable, black box (Rudin, 2019), or a hybrid between the two. The properties of interpretable models refer to transparency, which provides insight into how the model works (Lipton, 2018), this category includes decision trees such as CART and Random Forest and rule-based learners such as M5 and CUBIST. In contrast to the interpretable models, black-box models prioritise predictive accuracy over transparency and explainability (Mehdiyev et al., 2025). This category includes Support Vector Machine (SVM), Gradient Boost Machines (GBM), and Long-Short Term Memory (LSTM).
 - Models can be further classified into one of five categories based upon their approach to 'learning': '*supervised learning*'; '*unsupervised learning*'; '*semi-supervised learning*'; '*self-supervised*'; and '*reinforcement learning*' (Zhou, 2012).
 - In addition to the type of model, implementation details pertaining to which programming language (R, Python) and machine learning frameworks and tools (TensorFlow, PyTorch, and R packages) have been employed including the associated software dependencies, version numbers, and environmental configurations necessary for model training and deployment.
 - The sampling rates of the sensors and actuators, as well as the temporal aggregation methods employed for the inputs to the ML soft sensor, are also documented. To illustrate, data initially recorded at [original time interval] was resampled to [new time interval] using [aggregation method] (e.g., mean, sum) for sending to the inputs of the ML soft sensor.
- **Model architecture (*model_architecture*).** Terms include: *input_layer*, *layer*, *output_layer* that describe the specifics of the model's architecture.
 - For interpretable learners, the model architecture could include rules, linear regressions or decision trees, while the hyperparameters may include factors such as tree depth or regularisation parameters.

- For black-box models, such as neural networks, the model architecture refers to the number of layers, neurons, and activation functions, while the hyperparameters include the learning rate, batch size and optimiser. However, this can be model dependent, for instance, Support Vector Machines (SVM) do not have a traditional layered architecture but they do depend on the choice of support vectors and kernel type (e.g., linear, polynomial, radial basis function, etc.). Hyperparameters for SVMs include the regularisation parameter (C), kernel-specific parameters and the boundary conditions that determine which data points become support vectors.
- **Training strategy (*training_information*).** The training and testing strategy include details of the partitioning of the data set, the evaluation criteria and the validation approach. Terms include: *number_of_instances*; *hyperparameters*; *validation*; and *experiments_ID*.
 - This includes specifying the proportion of data allocated to the training and test sets, as well as any cross-validation strategies used to ensure the robustness of the models. Information on the evaluation metrics used during these phases, such as Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Mean Square Error (MSE) or coefficient of determination (r^2), is also documented.
 - In addition, information is provided on the stratification techniques used to ensure reproducibility and consistency of the dataset splits.
- **Model input (inputs) and output (outputs).** ML soft sensors have been built to accept three types of inputs: (physical or virtual) sensors, actuators, or computed variables. The input and output metadata provides a description of the features, their units, any time lag, etc. In the context of data preprocessing, the metamodel provides information regarding the application of feature scaling or temporal aggregation. Feature scaling refers to the techniques employed to standardise or normalise variables, thereby ensuring that they are on a comparable scale. This step is of particular importance when the variables have different units of measurement, ranges, or orders of magnitude (Metcalf et al., 2025) because, without standardisation, features with larger scales can dominate the learning process and degrade the performance of black-box models such as SVM or neural networks (de Amorim et al., 2023; Pinheiro et al., 2025). For example, temperature might be expressed in Kelvin (K) or Celsius (C), dissolved oxygen concentration in milligrams per litre (mg L^{-1}), or sugar flow rate in litres per hour (L h^{-1}).

2.4.3 Example models

Model	Model	Model	Programming	Package	Exporte	Use case	Reference
-------	-------	-------	-------------	---------	---------	----------	-----------

type	sub-type		Language	dependencies	d model format	description	
Interpretable soft sensor	Decision tree	CART	R	Name: rpart Version: 4.1.23	.rds	The inputs and output were not normalised. The soft sensors were constructed using a 10-fold cross-validation process with three repetitions applied to 89800 instances. Model performance was evaluated using MAE, SME, RMSE, and R ²	Acosta-Pavas et al., 2024
			Python	Name: sklearn.tree Version: 1.5.2 Class: DecisionTreeRegressor	.pkl	Experimental setup (data, hyperparameters, validation, etc..) replicated from Acosta-Pavas et al., 2024	<i>This paper</i>
		Random Forest	R	Name: ranger Version: 0.16.0	.rds	The inputs and output were not normalised. The soft sensors were constructed using a 10-fold cross-validation process with three repetitions applied to 89800 instances. Model performance was evaluated using MAE, SME, RMSE, and R ²	Acosta-Pavas et al., 2024
			Python	Name: sklearn.ensemble Version: 1.5.2 Class: RandomForestRegressor	.pkl	Experimental setup (data, hyperparameters, validation, etc..) replicated from Acosta-Pavas et al., 2024	<i>This paper</i>
	Rule-based learner	M5	R	Name: RWeka Version: 0.4.46	.rds	The inputs and output were not normalised. The soft sensors were constructed using a 10-fold cross-validation process with three repetitions applied to 89800 instances. Model performance was evaluated using MAE, SME, RMSE, and R ²	Acosta-Pavas et al., 2024
		CUBIST	R	Name: Cubist Version: 0.4.4	.rds	The inputs and output were not normalised. The soft sensors were constructed using a 10-fold cross-validation process with three repetitions applied to 89800 instances. Model performance was evaluated using MAE, SME, RMSE, and R ²	Acosta-Pavas et al., 2024
Black box soft sensor		SVM	Python	Name: sklearn.svm Version: 1.5.2 Class: SVR	.pkl	A hyperparameter grid was defined, including kernel types (linear and radial basis function - RBF), regularisation parameter (C) values 0.1, 1, 10, 100, epsilon values 0.01, 0.1, 1, 5. The model was fitted using 10-fold cross-validation for each of the 64 hyperparameter candidates, totalling 640 fits. The best hyperparameters identified through this process were kernel RBF, regularisation parameter C = 1 and epsilon = 0.01	<i>*This paper</i>
		GBM	Python	Name: sklearn.tree Version: 1.5.2 Class: GradientBoostingRegressor	.pkl	A hyperparameter grid was defined, including the number of boosting stages (100, 200, 300), learning rate values (0.01, 0.1, 0.2), and the maximum depth of individual trees (3, 5, 7). The model was fitted using 10-	<i>*This paper</i>

						fold cross-validation for each of the 27 hyperparameter candidates, resulting in a total of 270 fits. The optimal hyperparameters were identified as follows: 300 boosting stages, a learning rate of 0.1, and a maximum tree depth of 5. Since the weak learner in the GBM is a decision tree (CART), normalization is not necessary because decision trees are not sensitive to the scale of the input features	
		LSTM	Python	Name: tensorflow.keras.layers Version: 2.18 Class: LSTM	.keras	Data was reshaped to align with the requisite input format, with each sample comprising a single time step and multiple features. An empirical test was conducted to assess the performance of the LSTM model. The model comprised a two-layer LSTM network, with 256 units in each layer. This was followed by a dropout layer (with a rate of 0.2) to mitigate overfitting, and batch normalization to enhance training stability. An additional fully connected layer with 256 units and a final output layer were included to predict the penicillin concentration. The model was compiled using the Adam optimizer with a learning rate of 0.001 and trained for 23 epochs, using early stopping to avoid overfitting. The best-performing model, based on validation loss, was saved.	<i>*This paper</i>

Table 2. Summary of the implementation details for the interpretable soft sensor. Soft sensor examples including the programming language, dependency packages and exported model formats. Additional ML models - denoted by * - were developed using the identical dataset (training and test) and identical predictors and target variables as those considered in (Acosta-Pavas, Robles-Rodriguez, et al., 2024).

To showcase the functionality of both STAMM and the ML repository, a number of soft sensors in industrial-scale fed-batch penicillin fermentation processes that have been developed previously (Acosta-Pavas, Robles-Rodriguez, et al., 2024) were converted to be usable in STAMM. These models primarily focused on the use of interpretable machine learning (ML) models for predicting penicillin concentration, including decision trees such as CART and Random Forest, rule-based learners such as M5 and CUBIST (**Table 2**). These four soft sensors were constructed in R; therefore, to demonstrate model interoperability the CART and Random Forest models were implemented in Python (**Table 2**). Three additional ML soft sensors (**Table 2**) one representing a Support Vector Machine (SVM), another a Gradient Boost Machine (GBM), and the final a Long-Short Term Memory (LSTM) were also implemented using the same training and evaluation dataset. Each of these models are representative of the black-box variety showcasing that STAMM is model agnostic. To develop these three models the train-test split ratio was set to 80:20 batches for each control strategy and the models used identical predictors, target variables and train-test strategy as those considered in Acosta-Pavas et al. (2024). **Table 3** summarises the classical performance metrics obtained for each machine learning model after training and evaluation on the designated test set. For each learner, statistical criteria including the coefficient of

determination (R^2), mean squared error (MSE), root mean squared error (RMSE), and mean absolute error (MAE) are reported. Although the primary objective of this study is not to emphasise the performance of the black box soft sensors, **Table 3** shows that LSTM achieved superior predictive accuracy compared to the interpretable models. LSTM model obtained the best overall performance with an R^2 of 0.916, the lowest MSE (8.800), RMSE (2.966), and MAE (1.843). This is followed closely by the CUBIST model ($R^2 = 0.908$), GBM ($R^2 = 0.904$), and RF ($R^2 = 0.905$), all of which show strong performance across all statistical metrics. In contrast, classical learners based on rules or decision trees, such as CART and M5 achieved lower R^2 scores (0.775 and 0.777, respectively) and higher error values, with CART showing the weakest performance (MSE = 24.061, RMSE = 4.905). These differences highlight the advantage of black box models in capturing the complex, nonlinear, and temporal dynamics of the fermentation process, which are often difficult to represent with simpler rule-based or tree-based learners. This performance gap reinforces the well-known trade-off between interpretability and accuracy in machine learning applications.

Learner	Statistical criteria			
	r^2	MSE	RMSE	MAE
CART	0.775	24.061	4.905	3.328
M5	0.777	23.823	4.880	3.221
CUBIST	0.908	9.916	3.149	1.920
RF	0.905	10.202	3.194	1.928
SVM	0.847	16.107	4.013	2.317
GBM	0.904	10.146	3.185	1.930
LSTM	0.916	8.800	2.966	1.843

Table 3. Results of the ML soft sensors evaluated with a test set. Statistical criteria used to estimate the performance of regression learners: coefficient of determination (r^2); mean squared error (MSE); root mean squared error (RMSE); and mean absolute error (MAE). Results of the interpretable learners as CART, M5, CUBIST and RF and strategy for splitting the test and training sets were presented in (Acosta-Pavas, Robles-Rodriguez, et al., 2024). Un-shaded cells represent interpretable learners whilst shaded cells represent black-box models, bold values represent the 'best' values for each model type and statistical criteria.

2.5 Dashboard

2.5.1 Relevance

For the model output in STAMM to be usable it must be presented to a user for example, a model developer, laboratory technician, experimentalist, or user. By supporting

experimentation processes and monitoring key industrial parameters within a framework based on ML soft sensors, a dashboard provides visualisations for decision support in an I4.0 environment to be leveraged by operators, engineers, and researchers. In today's data analytics, the ability to interpret and integrate data effectively is a crucial skill for both experts and non-experts' users. This skill is particularly significant in organizations, research facilities, and other environments where uncovering patterns, predicting outcomes, and supporting decision-making must be achieved smoothly and intuitively. Specialized dashboards serve as a key solution, acting as a bridge between raw data and actionable insights (Hjelle et al., 2024). These tools provide an interactive platform that consolidates and organizes data from multiple sources, presenting key metrics, trends, and insights in an easily accessible format. STAMM required an interactive dashboard that integrated simulations generated by ML soft sensors as well as displaying the real-time, input data. By enabling real-time monitoring and analysis, a dashboard can empower a user to identify patterns and make informed decisions quickly (Nasir et al., 2024). By supporting experimentation processes and monitoring key industrial parameters within a framework based on ML soft sensors, a dashboard provides visualisations for decision support in an I4.0 environment to be leveraged by operators, engineers, and researchers. In addition, it enables enhanced traceability, reproducibility, and optimisation of machine learning models while facilitating the interpretation of model outputs and their alignment with the industrial process.

Amongst the various graphical user interfaces (GUI) available for creating dashboards in Python (see <https://wiki.python.org/moin/GuiProgramming>), Plotly and Dash have emerged as popular solutions for building user-friendly, cross-platform, real-time interfaces that require minimal programming expertise in web development and deployment (Schroeder et al., 2022). This feature facilitates the containerization of applications, enabling deployment across diverse environments, including local machines, testing servers, or real-time production systems. Furthermore, Dash offers an extensively customizable - in both layout and functionality - Python-based framework that integrates seamlessly with advanced web development technologies such as React, JavaScript, Flask, CSS, and HTML. It is this flexibility allowing broader customization which differentiates it from other dashboard tools such as Tableau (Meier, 2023) or Power BI (Powell, 2018) that rely heavily on visualizations that are predefined and a stratified user interface. Therefore, Dash was chosen for STAMM because it simplifies the dashboard creation process into two primary steps: designing the visual interface and implementing actions or triggers for dynamic visualizations with specialized controls.

2.5.2 STAMM's dashboard general structure

The STAMM Dashboard is an interactive web application developed with Plotly and Dash, designed to support both process control engineers and machine learning specialists working with soft sensors in industrial production environments. It provides a unified interface for visualizing real-time process data, managing model deployments, and evaluating soft sensor performance under operational conditions (for a practical, step-by-step explanation of how to

navigate the dashboard, a dedicated user guide is available at <https://stamm.inrae.fr>). The dashboard comprises four main sections, each dedicated to a specific aspect of soft sensor supervision and analysis (see **Figure 5**).

2.5.3 STAMM dashboard: data source

The Data Source section centralizes all information required by the STAMM Dashboard and provides access to the data associated with ongoing or completed experiments (with data supplied by the Data Producer, see **Section 2.2**). It provides detailed statistics for all recorded variables, including sensors, actuators, and computed variables, enabling users to rapidly assess the distribution, scale, and characteristics of the experimental data (**Figure 5a**).

- The *Experiment Data Browser* enables users to choose the experiment they want to explore, whether currently running or previously completed. Once an experiment is selected, the dashboard displays a structured summary of all associated variables, including type, identifier, measurement units, and descriptive statistics such as mean, maximum, and minimum values. Selecting an experiment in this module also acts as a prerequisite for executing soft sensor models provided in the next section of the dashboard.
- The *Recent Measurements* functionality allows inspection of the most recent data collected from ongoing experiments. Users can define a temporal window to retrieve measurements received within a specified interval. For each experiment, key information such as start and end times, timestamp of the latest measurement, and the total number of recorded samples is presented, supporting immediate verification of data availability.
- The *Data Point Distribution* panel provides a quantitative view of data completeness and quality across a chosen time range. Users can select one or multiple variables, or all variables associated with the experiment, and the dashboard displays the total number of valid data points collected during the interval. This visualization helps identify potential gaps or irregularities in data acquisition

Data source

Experiment Data Browser:

Project Name:

Statistical summary of the variables from experiment: Batch 5

Type	Name	Unit	Mean	Max	Min
Sensor					
Actuator					

Recent Measurements Value: Unit:

Experiment ID	Start time	End time	Batch size
Batch 5			
Batch 7			

Data Point Distribution Value: Variable:

Soft sensors

Model name:

Interval update (seconds):

Model metadata: LSTM [python version]

```

name: "LSTM"
version: "v.1.0"
id: "0009 [python]_penicillin_lstm"
authors: "Garcia, C., Astudillo A., Metcalfe B., Koeberst J.J., Castillo E. & Corrales, D. C"
status: "online"

```

Variable: Name:

Experiment ID: Batch 5

(a) **Data source view.** Here, the experiment ID to which the soft sensor will be connected is selected. Additional information regarding the number of data points generated in the experiment is also displayed.

(b) **Soft sensors view.** In this view, the model to be deployed is selected for the experiment ID (Batch 5) chosen in the Data source view.

Monitoring

Deployed model: Drift detector:

Experiment ID: Batch 5 Time range selection (data points):

Metric parameters: Variable: Drift detector parameters: θ ϵ

Explanation:

Drift result:

Density plot of training set and Experiment ID: 5 – Variable: DO₂

Simulation assessment

Deployed model: Experiment ID: Batch 5

Time range selection (data points):

Variable: Name:

Experiment ID: Batch 5

Variable: Enter a value:

Time	DO2	LSTM simulations	Report data point
			Report
			Report

(c) **Monitoring view.** The density plot compares the data used to train the model with data from the connected experiment (Batch 5) for a specific model variable (DO₂).

(d) **Simulation assessment view.** In the 'Report data point' column, a popup can be opened to label a data point in case any abnormality occurred.

Figure 5. Mock-ups of the main dashboard views. Mock-up of the dashboards: (a) data source, (b) soft sensors, (c) monitoring, and (d) simulation assessment (see Supplementary Section 1). For the dashboard user guide, visit <https://stamm.inrae.fr>

2.5.4 STAMM dashboard: soft sensors

The Soft Sensors module provides a centralized workspace for deploying model-based predictions in real time. Users can select any soft sensor available in the Model Registry (Section 2.4) to operate alongside an ongoing experiment, allowing for simultaneous comparison of predicted outputs with measured process variables, including sensors and actuators. This integrated view supports model validation, performance assessment, and the tracking of key indicators during experimental operation (Figure 5b).

The selection of a model is followed by the presentation of detailed metadata, including configuration parameters, version, input and output structure, hyperparameters, and operational status. This information provides transparency regarding the model's implementation and facilitates reproducibility of predictions. Users may also review historical or real-time predictions for the experiment currently chosen in the Data Source section (Section 2.5.3), with the option to include additional process variables in the visualization to support comprehensive analysis.

The Soft Sensors module features interactive visualizations that update at configurable intervals, enabling continuous monitoring of process dynamics and model performance. Users can adjust the displayed time window to focus on short-term trends or examine longer-term behavior, from hours to months, providing flexibility in the assessment of model predictions and experimental outcomes. By integrating predictions and experimental data in a single interface, this module enhances the interpretability of soft sensor outputs and supports informed decision-making during experimental campaigns.

2.5.5 STAMM dashboard: monitoring

The Monitoring view provides a comprehensive functionality for the continuous evaluation of deployed soft sensors, integrating both *Data Drift Detectors* and *Model Divergence* analyses. These complementary functionalities enable users to track model performance, run drift detectors, and compare outputs from different models, thereby supporting informed decision-making during ongoing experiments.

Data Drift Detectors operate at both univariate and multivariate levels, offering real-time assessment of changes in input distributions relative to the data used during model training. Visualizations, including train-test density comparison plots, allow users to inspect differences between current operational data and the original training datasets, facilitating early detection of data shifts or potential process anomalies (Figure 5c).

By combining drift detection with model divergence analysis, the Monitoring module enables systematic assessment of soft sensor reliability, ensuring that predictions remain accurate and robust throughout the course of an experiment. This integrated approach supports proactive maintenance of model integrity and enhances confidence in real-time decision-making based

on soft sensor outputs. Given the relevance of this functionality, **Section 2.6** is dedicated to a conceptual explanation of the *Data Drift Detectors* and *Model Divergence* analyses.

2.5.6 STAMM dashboard: simulation Assessment

The Simulation Assessment module provides an offline environment for post-run evaluation by presenting recorded sensor and actuator measurements together with the corresponding soft sensor simulations. Within this view, users can inspect the time series in detail and label specific data points that appear erroneous, inconsistent with expected process dynamics, or indicative of sensor malfunction or model misprediction (**Figure 5d**). This annotation capability enhances data traceability and supports the creation of high-quality, curated datasets. Beyond immediate diagnosis, the labeled data may serve as a valuable resource for downstream tasks, such as model refinement or retraining, enabling continuous improvement of soft sensor performance. This functionality contributes to the human-in-the-loop approach discussed in **Section 2.6.4**, where expert inspection and annotation play a central role in validating soft sensor behaviour and identifying unexpected process dynamics

2.6 - Monitoring of ML soft sensors

2.6.1 Monitoring

Once deployed, ML soft sensors require continuous monitoring and maintenance to ensure their continued reliability and sustained performance under real-world conditions. Unlike traditional models evaluated only on static test sets, data-driven models are exposed to dynamic environments where process conditions, data distributions, and control strategies may evolve over time. In the absence of adequate monitoring, these alterations have the potential to result in degradation or deterioration of the model resulting in a misalignment between input parameters and intended outcome, a reduction in accuracy, and a diminution of confidence in the predictions made. There are two main causes of 'drift', the first being data drift which refers to a change in the distribution of input features compared to the original training and validation datasets and second is concept drift, which occurs when the underlying relationship between inputs and outputs evolves over time, causing the model's learned mapping to no longer accurately reflect the true process dynamics (Gama et al., 2014). Data drift may occur for instance, when new bioprocessing experiments go beyond the limits of previous experiments resulting in values outside of the original model input ranges. Whereas, the second type of drift, concept drift relates to changes in the relationship between input features and the target variable, this may occur when new strains are used in fermentation experiments leading to a decoupling between input feature and variable.

Therefore, STAMM requires a module that assesses the continued confidence of the model by monitoring changes in data behaviour. Monitoring of ML soft sensors ensures the identification and response to changes in data or process dynamics with the methodology often referred to as 'drift detection' or 'change detection'. These methods determine and identify a specific time instant or time interval when a change arises in the properties of the

641 target object (Basseville & Nikiforov, 1993) or determine the time constraints on the detection
642 delay, with the objective of enabling the learner to adapt to the change efficiently and to ensure
643 high performance (Pears et al., 2014).

644 2.6.2 Drift detectors and STAMM

Type	Metric	Description	Equation	Interpretation
Univariate	ADaptive WINdowing (ADWIN)	ADWIN employs an efficient method for maintaining a variable-length window of recent items, ensuring that no changes have occurred in the data distribution (Bifet & Gavaldà, 2007).	ADWIN uses the Hoeffding bound to examine the change between the means, μW_0 and μW_1 , on two sub-windows W_0 and W_1 : $ \mu W_0 - \mu W_1 > 2\epsilon_{cut}$ Where ϵ_{cut} is the optimal cut: $\epsilon_{cut} = \sqrt{\frac{1}{2m}} \cdot \ln \frac{4}{\delta}$ Where m is the harmonic mean of the two windows, and δ is a pre-defined confidence parameter	<i>If $\mu W_0 - \mu W_1 > 2\epsilon_{cut}$ then:</i> The change exceeds expected variability, indicating a significant shift in the data distribution. <i>If $\mu W_0 - \mu W_1 \leq 2\epsilon_{cut}$ then:</i> The change is within expected variability, and the distribution is considered stable. * The δ parameter can be predefined according to the following recommendations. $\delta = 0.05$, which corresponds to 95% confidence and is commonly used; $\delta = 0.01$, which corresponds to 99% confidence and is more conservative; and $\delta = 0.1$, which corresponds to 90% confidence and makes the detector more sensitive.
	Population Stability Index (PSI)	This quantifies covariate shifts between two distributions (Taplin & Hunt, 2019).	$PSI = \sum_{i=1}^n \left((r_i - m_i) \cdot \ln \left(\frac{m_i}{r_i} \right) \right)$ where r_i and m_i : $r_i = \frac{\text{number of reference data points in bin } i}{\text{number of reference data points}}$ $m_i = \frac{\text{number of monitored data points in bin } i}{\text{number of reference data points}}$	$PSI < 0.1$ no significant population changes $0.1 \leq PSI < 0.25$ moderate change to the distribution $PSI \geq 0.25$ suggests significant changes to the distribution
Multivariate	PCA-based change detection (PCA-CD)	This method employs a principal component analysis (PCA) to identify shifts in multidimensional data streams (Qahtan et al., 2015). PCA-CD uses two sliding windows W_0 and W_1 .	W_0 represents the reference window, and W_1 the current window. W_0 and W_1 are projected onto the top k principal components obtained from W_0 $y = V_k^T(x - \mu_0)$ For each principal component i , the method estimates the 1-D density $pW_0(Y_i)$ and $pW_1(Y_i)$ from the projected data. Changes are quantified via symmetric Kullback-Leibler (KL) divergence: $D_i = KL(pW_0(Y_i) pW_1(Y_i)) + KL(pW_1(Y_i) pW_0(Y_i))$ The maximum divergence across PCs defines the overall change score: $S = \max (D_i)_{i=1..k}$	S quantifies the strongest change between W_0 and W_1 . Detection: compare $S = (D_1, D_2, \dots, D_k)$ to a threshold β . Exceeding β indicates that a change occurred. Interpretation: S alone does not identify the type of change. To determine whether the change is in mean, variance, or correlation, inspect the principal component i^* responsible for S and analyse how its distribution differs between W_0 and W_1 : - Shift in mean: mean change - Change in spread: variance change - Change in shape: correlation change
	KDQ-tree detection	This method partitions the feature space recursively using a combination of k-d tree and quantile-based splits. Two windows are maintained: a reference window and a current window. At each leaf (cell) of the tree, the algorithm stores counts of samples from each window. Drift is detected by measuring the divergence between the empirical distributions of both windows over all KDQ-Tree cells (Dasu et al., 2006).	W_0 represents the reference window, and W_1 the current window. For each KDQ-Tree leaf i : $P_i = \frac{n_i^0}{ W_0 }, Q_i = \frac{n_i^1}{ W_1 }$ Where: - n_i^0 is the number of samples from the reference window in cell i - n_i^1 is the number of samples from the current window in cell i - $ W_0 $ and $ W_1 $ are total number of samples in each window	<i>If $D_{KL} < \theta$ then:</i> Both windows follow the same distribution, counts in each KDQ cell remain similar <i>If $D_{KL} > \theta$ then:</i> the current data occupy the space in a substantially different pattern compared to the reference period indicating that the underlying multivariate distribution has changed

			The divergence is computed as: $D_{KL}(P Q) = \sum_{i=1}^L P_i \log\left(\frac{P_i}{Q_i}\right)$ Drift is detected when: $D_{KL}(P Q) > \theta$ Where θ is a detection threshold	
--	--	--	---	--

Table 4. Summary of the drift detection techniques implemented in a standalone Python package named 'drift detectors pack' (https://gitlab.com/stamm-4m/drift_detectors_pack). This package enables the integration and execution of multiple drift detection techniques, including both univariate and multivariate approaches

To support this functionality STAMM employs various univariate and multivariate drift detectors to detect concept and data drift using a dedicated, stand alone, Python package that has been developed alongside STAMM, named 'drift detectors pack'. This package enables the integration and execution of multiple drift detection techniques, including both univariate and multivariate approaches (**Table 4**). Thereby, allowing the implementation of a robust and customisable monitoring pipeline with methods including statistical data drift detectors as well as model divergence strategies (**Section 2.6.3**) for deployed ML soft sensors. Furthermore, it facilitates human-in-the-loop inspection of soft sensor predictions, including the detection of anomalies, sensor faults, and unexpected process dynamics through interactive visualisation. STAMM partitions the data stream into a sliding window based on either data size or time interval with this performance being compared with that of a reference window (Gama et al., 2014) using univariate statistics (**Table 4**).

2.6.3 Model divergence

In addition to drift, STAMM also determines the degree to which there is 'model divergence', or 'model disagreement', between predictions or outputs from multiple models when applied to the same input data (Uma et al., 2022). STAMM evaluates the predictions generated from the deployed soft sensor models to understand the extent of prediction differences (model divergence) and to evaluate whether this variability can be attributed to concept drift (**Figure 6**).

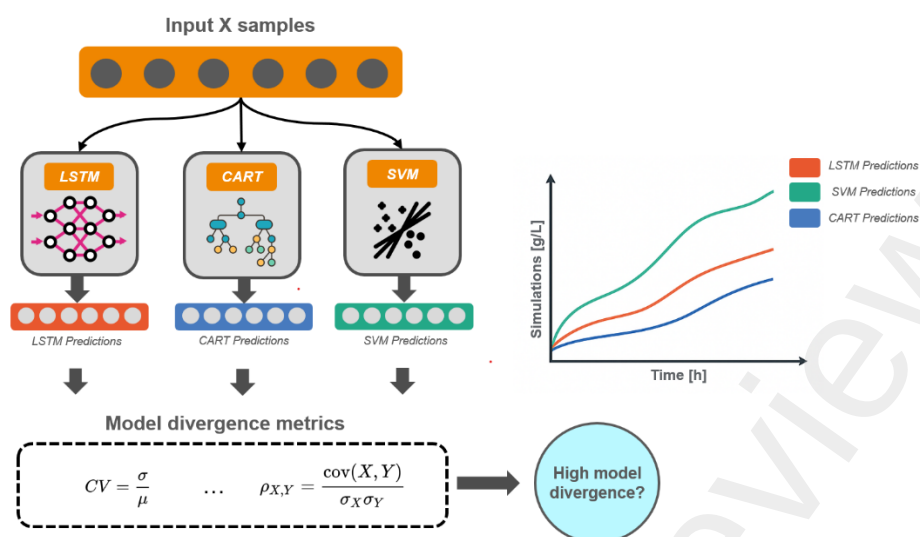


Figure 6. Performance estimators based on model disagreement metrics. Predictions generated by a set of ML soft sensor models are analysed using model disagreement metrics to evaluate potential concept drift. In this example, three soft sensor models, trained on the same dataset and predicting the same target variable are evaluated. All models receive the same input data. Within the context of STAMM, let us assume that the model deployed in real time is an LSTM. The goal of this analysis is to compute a divergence metric between the deployed LSTM and the alternative models stored in the Model Registry by computing a pairwise analysis: LSTM (deployed model) vs. SVM; LSTM (deployed model) vs. CART. These pairwise comparisons allow the system to quantify how much the deployed model deviates from other models that have been validated on the same data.

Model divergence is quantified using the dispersion of outputs across models, thereby capturing prediction inconsistency (Zhou, 2012; Breiman, 1996). Pairwise differences, such as the mean squared error between predictions from different models, are used to quantify the degree of similarity (Kuncheva & Whitaker, 2003). Whilst the correlation coefficients between models' outputs assess the relationship with lower correlation coefficients indicative of higher diversity (Dietterich, 2000). Finally, ranking consistency via Spearman's rank correlation evaluates prediction order similarity, which is often useful when prioritisation is key (Opitz & Maclin, 1999) an aspect that can be critical for automated alerting systems.

2.6.4 Human in the loop, inspection of soft sensor simulations and unexpected process dynamics

Once STAMM computes both drift (Section 2.6.2) and model divergence (Section 2.6.3) these values are displayed to a user (see Section 2.5.6). This focus on a collaborative human-in-the-loop approach, as opposed to automatically performing an action, ensures that user needs and goals are aligned with STAMM whilst also ensuring transparency and accountability. The user can interpret whether the observed differences are expected due to variability in model architecture, sensitivity to noise, or model capacity, or whether the results indicate a genuine model performance issue. When the divergence becomes substantial, corrective actions such as retraining, model recalibration, or even switching the deployed model may be required.

Likewise, the *Simulation Assessment* (functionality introduced in **Section 2.5.6**) tab in the dashboard overlays raw sensor data, actuator setpoints, and soft sensor predictions within an interactive plotting interface allowing users to inspect the time series; detect temporal correlations; identify lags; and potential discrepancies among data elements. Users can manually annotate data, flagging individual data points and model output within an interactive table and tagging anomalous behaviours, such as physically implausible readings (e.g., temperature values exceeding the operating tolerance of a sensor); soft sensor predictions that exhibit unrealistic dynamics (e.g., negative concentrations); faults in hardware sensors and soft sensors alike.

By manually inspecting the physical and virtual output offers an additional benefit: the detection of process behaviours not previously considered during the system's design or modelling. These behaviours, which may not correspond to noise or error, can reveal emergent dynamics, equipment degradation patterns, or operational regimes that fall outside nominal process conditions. As such, the *Simulation Assessment* component of STAMM supports not only validation and fault detection, but also exploratory monitoring, providing a foundation for deeper process understanding and continuous improvement.

3. Discussion

3.1 STAMM

Presently, while there exist mature open-source MLOps platforms such as Kubeflow (<https://www.kubeflow.org>), MLflow (<https://mlflow.org>), and Metaflow (<https://metaflow.org>), these systems are primarily designed for general-purpose machine learning workflows and are not tailored to the unique demands of ML-based soft sensors in real-time industrial environments such as continuous monitoring, concept drift detection, and integration with process control systems. This creates a gap between general-purpose MLOps solutions and the specialized needs of soft sensor deployment under Industry 4.0 principles (i.e., interoperability, information transparency, technical assistance, and decentralization of decisions). Previous work such as (Cao et al., 2025) introduced the *Soft Sensor Manager*, a Python based prototype with automated feature extraction and “one-click” online deployment. While it demonstrates a well-thought-out research prototype, its practical applicability in industrial settings is limited being (i) not (currently) publicly available for reuse, and (ii) although it mentions version control and online deployment, it lacks detailed mechanisms for managing model, data, configuration versions, deployment across industrial infrastructure, drift detection, or automated maintenance. These gaps highlight the need for an open, modular, production-ready framework for ML-based soft sensors that provides full lifecycle management, real-time monitoring, and integration with process control systems.

Therefore, the primary contribution of this paper has been the development of an open source, lightweight yet extensible, minimum viable prototype (MVP) of a Machine Learning Operations (MLOps) application, referred to as STAMM. STAMM is focused on supporting the needs of

both users, developers, and the end-to-end lifecycle of ML soft sensors, allowing a user to deploy, monitor, and maintain a ML soft sensor in a 'low code' environment. For developers this framework allows for the construction of reproducible, modular workflows deployed using Apache Airflow. Allowing for the automation and scheduling of each component involved in soft sensor integration from real-time data retrieval from a time series database; prediction; monitoring; and the exporting and storage of any output.

Although the platform has been developed in the Python programming language it supports models in both Python and R enabling the seamless integration of externally trained models regardless of the development environment. This design choice enhances the modularity and interoperability of the framework, allowing data scientists to adopt the modelling tools that best fit their domain expertise while still benefiting from the unified deployment and monitoring infrastructure provided by STAMM. Central to this is STAMM's incorporation of both a (i) ML repository and (ii) structured metadata to describe each ML soft sensor. These metadata files enhance traceability, reproducibility, and integration by systematically capturing model details, preprocessing methods, input/output variables, and implementation parameters. This approach aligns with FAIR - Findable, Accessible, Interoperable, Reusable – principles (Barker et al., 2022; Wilkinson et al., 2016) supports consistent integration into MLOps pipelines.

Complementing the backend components is the frontend, interactive, dashboard that provides real-time visibility into soft sensor performance, parameter control, and drift detection alerts. This dashboard promotes collaboration across user groups and facilitates anomaly detection through accessible, domain-specific visualisations. Unlike general-purpose tools like Tableau or Power BI, STAMM's dashboard is tailored to the complexity of industrial data and research workflows, providing greater flexibility and integration depth.

3.2 Modular components

STAMM is designed with a modular architecture that enables seamless integration and interchangeability of data sources, processing modules, and monitoring components. Core system elements including time-series databases, service orchestrators, and real-time visualization modules can be connected, replaced, or reconfigured with minimal effort. This flexibility allows STAMM to adapt rapidly to diverse industrial processes, evolving experimental conditions, or new analytical requirements.

The modular aspect of STAMM, in which various layers have been implemented as standalone Python packages, ensures that not only can components be 'swapped out' in future or for specific use cases but that these components can be updated independently and/or used elsewhere. Allowing users to adapt the package to their needs it offers a flexible solution that can be tailored to specific industrial constraints, unlike proprietary software. For instance, the drift detection layer, is its own Python package that currently supports both univariate and multivariate drift detectors, as well as performance-based estimators, to enable robust post-deployment monitoring of model validity and data consistency. However, users and/or

774 developers can, if they wish, develop their own or incorporate different packages that deal with
775 detecting deterioration in model performance or set up their own specific monitoring
776 conditions. Through allowing users to adapt the package to their needs it offers a flexible
777 solution that can be tailored to specific industrial constraints, unlike proprietary software. A
778 capability that is essential to ensure the timely detection of changes in data or process
779 dynamics and to trigger appropriate model maintenance or retraining actions.

780
781 The platform's modular design also provides users with control over data governance and
782 storage choices. While InfluxDB is currently employed for high-frequency process data,
783 STAMM allows alternative database backends to be integrated, giving teams the flexibility to
784 select solutions that align with institutional policies, IT infrastructure, or performance
785 requirements. For industrial processes where continuous monitoring is critical, databases
786 optimized for time-series data such as TigerData (<https://www.tigerdata.com>) are
787 recommended to ensure efficient storage, querying, and retrieval. Importantly, the system's
788 architecture does not impose a specific solution, preserving operational freedom while
789 supporting reliable MLOps pipelines.

790 **3.3 Future work**

791 Future work is envisioned to build upon the current iteration of STAMM in several promising
792 directions that could enhance the robustness, adaptability, and versatility of ML soft sensor
793 deployment in industrial settings. First, future work will focus on integrating automated
794 mechanisms for incremental learning, enabling ML soft sensors to adapt continuously to
795 changes in process behaviour without requiring full retraining. The design of such mechanisms
796 must account for the nature of the underlying learning algorithm, for instance, the models built
797 in this study such as decision trees like CART and M5, rule-based models such as CUBIST,
798 ensemble methods like RF and GBM, or recurrent neural networks like LSTM, each of which
799 requires a distinct strategy for incremental updates. The incorporation of active learning
800 strategies also holds potential to prevent catastrophic forgetting while allowing the model to
801 selectively learn from new, informative instances. Secondly, the STAMM framework may be
802 extended to support dynamic and hybrid modelling approaches (Schweidtmann et al., 2024;
803 von Stosch et al., 2014). For example, the integration of dynamic models based on ordinary
804 differential equations (ODEs) would enable the representation of mechanistic process
805 knowledge as mass and energy balances, kinetic expressions, or computational fluid
806 dynamics (Gábor & Banga, 2015). In this context, hybrid models, which combine data-driven
807 techniques with first-principles models, can be particularly powerful for example, through using
808 ML models to estimate parameters or residuals in ODE-based systems. Such extensions
809 would broaden STAMM's applicability in domains requiring explainable and physically
810 grounded predictions. Finally, with respect to model development and use, a key area for
811 future development of STAMM is aiding in the expansion of ML interoperability by facilitating
812 models from other programming languages (e.g., MATLAB), enabling researchers to integrate
813 models built in programming environments they are used to while benefiting from STAMM's

unified infrastructure for deployment, monitoring, and maintenance. Further enhancing the modularity and accessibility of STAMM.

Future work is also planned to incorporate links to metadata catalogues such as FAIRDOM-SEEK (Wolstencroft et al., 2011, 2015, 2017). Beyond local or project-specific storage solutions, metadata catalogues provide a structured environment to organise and manage project-level information (Child et al., 2022), including experimental descriptions, protocols, equipment configurations, and analytical results (Wolstencroft et al., 2017). Such platforms facilitate standardisation, enhance traceability, and ensure metadata remains accessible independently of operational data pipelines. By separating high-level project information from real-time measurements, metadata catalogues complement monitoring and predictive systems while supporting reproducibility and knowledge transfer. As a tailored instance of FAIRDOM-SEEK for industrial contexts involving biotechnology, biomanufacturing, and bioprocessing the IBISBA Knowledge Hub (IBISBAkhub; <https://hub.ibisba.eu>) can unify detailed project descriptors, batch identifiers, experimental workflows, and associated documentation following established community standards. Acting as a metadata catalogue the IBISBAkhub, or any other instance of FAIRDOM-SEEK (Wolstencroft et al., 2017), would be able to store descriptors, operating modes, equipment configurations, etc. regarding batches that would guide model deployment and interpretation, and enrich analyses, diagnostics, and drift assessments. At the same time, STAMM would manage real-time and historical process data, soft sensor execution, and performance monitoring. Connecting STAMM to a metadata catalogue would create a coherent ecosystem, ensuring that operational monitoring is informed by curated project context, enhancing traceability, reproducibility, and collaborative analysis across teams and facilities.

Declaration of Competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data used in the testing and development of STAMM correspond to an industrial scale fed-batch fermentation for the production of Penicillin from the fungus *Penicillium chrysogenum*, published as Industrial Penicillin Simulator (IndPenSim) in doi: [10.17632/pdnjz7zz5x.1](https://doi.org/10.17632/pdnjz7zz5x.1) by (Goldrick et al., 2019, 2015).

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work several of the authors used ChatGPT (mainly GPT-4o) to rephrase sentences and check for language mistakes. After using this tool, the authors

reviewed and edited the content as needed and take full responsibility for the content of the publication.

Availability of source code and requirements

- **Project name:** Soft sensor monitoring and maintenance framework for Machine learning Models
- **Project home page:** <http://stamm.inrae.fr/>
- **Git repository:** <https://gitlab.com/stamm-4m>
 - **Dashboard:** <https://gitlab.com/stamm-4m/dashboard>
 - **Model registry:** <https://gitlab.com/stamm-4m/model-registry>
 - **Workflow management system:** <https://gitlab.com/stamm-4m/workflow-orchestrator>
 - **Drift detectors:** https://gitlab.com/stamm-4m/drift_detectors_pack
 - **Database:** <https://gitlab.com/stamm-4m/database>
 - **Node-red – fed-batch fermentation:** <https://gitlab.com/stamm-4m/indpensim>
- **Programming language:** Python
- **License:** Apache License, Version 2.0

Supplementary material

S.1. Dashboard views

S.1.1. Data source view

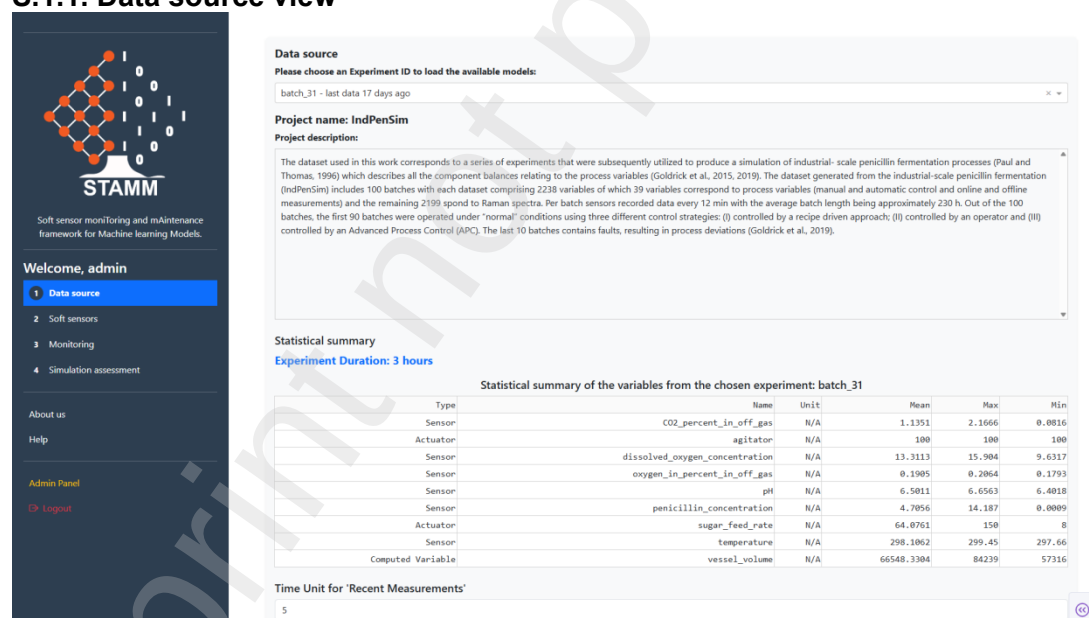
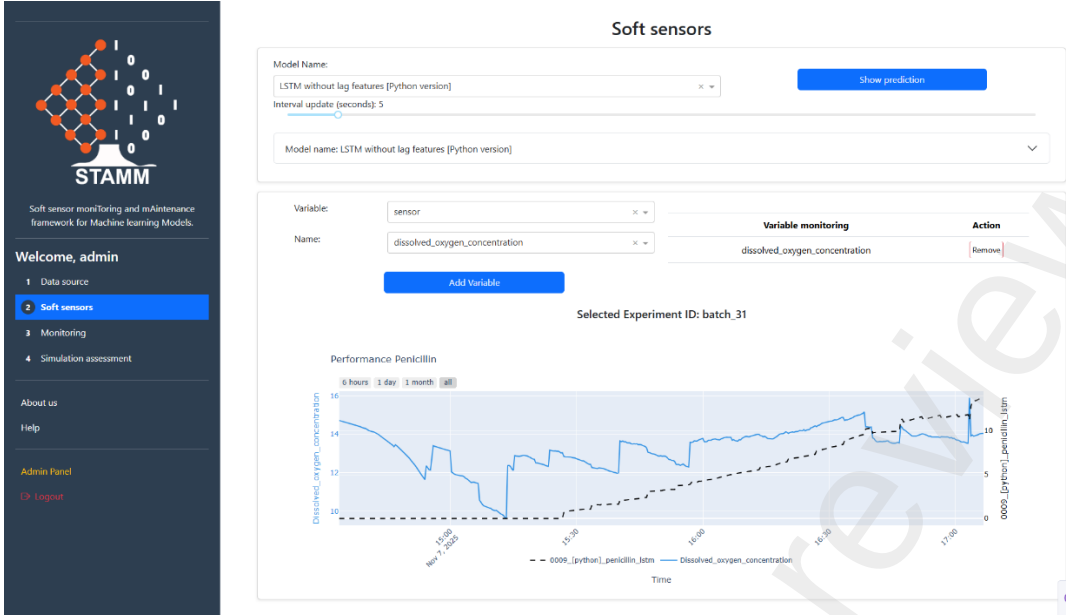


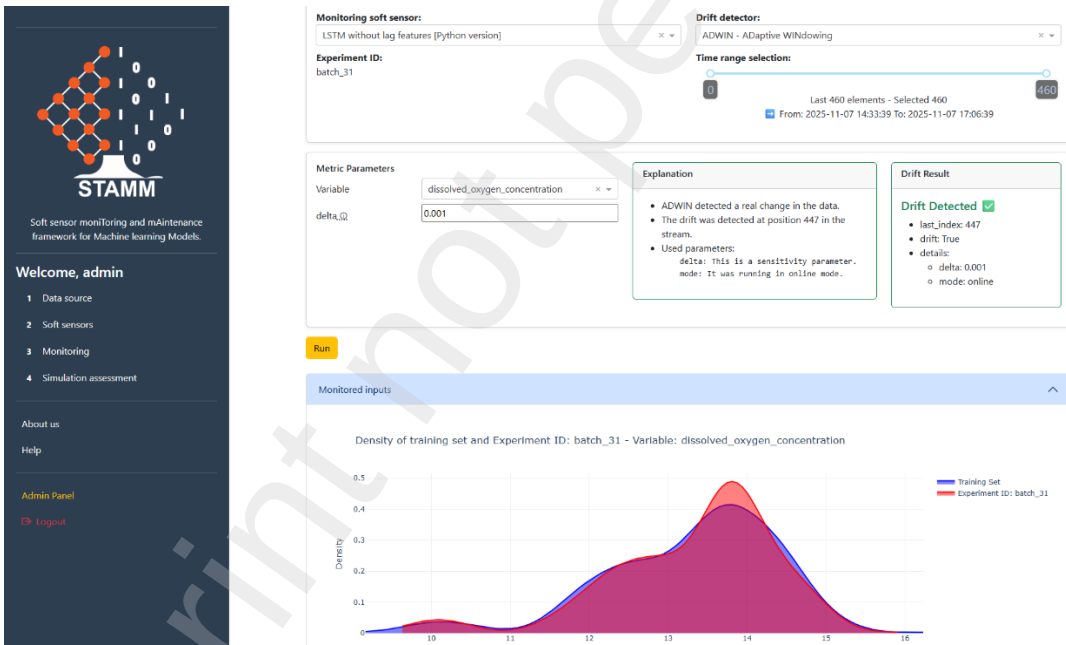
Figure S1. Data source view. On the left, the four main sections of the STAMM Dashboard are displayed. On the right, the selection of Experiment ID 5, which produced data ten days ago. The panel also presents a description of the project associated with the experiment, together with a table summarizing the data statistics generated by the corresponding sensors and actuators

874 **S.1.2. Soft sensors view**



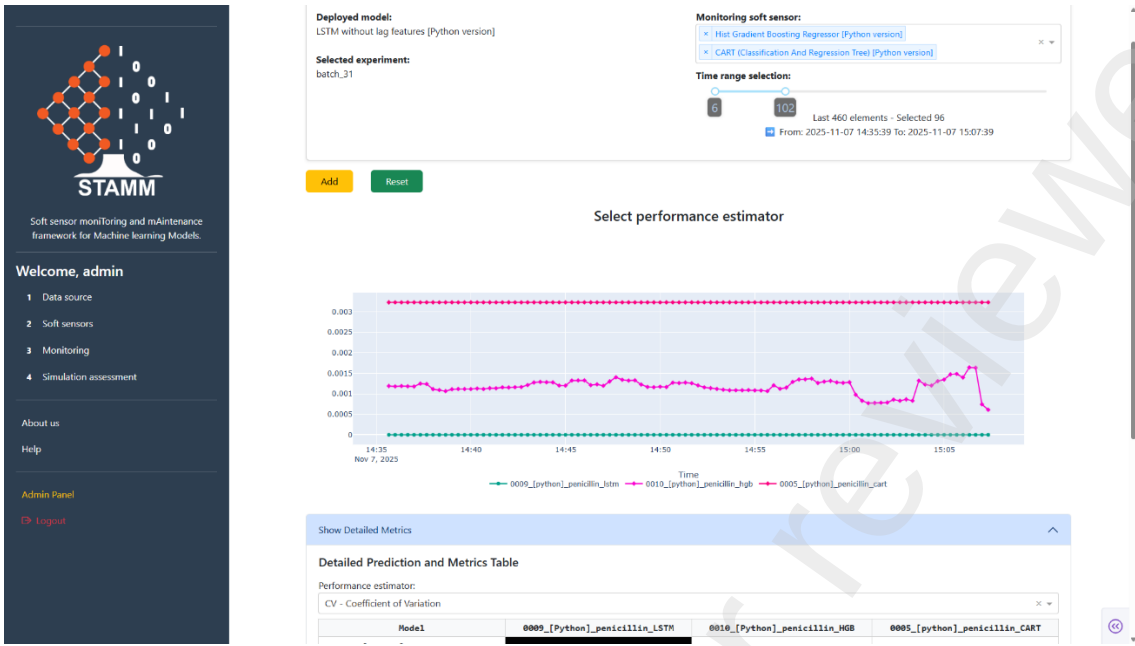
875 **Figure S2.** Soft sensors view. On the left, the four main sections of the STAMM Dashboard are displayed. On the right, the
876 selection panel allows users to choose a soft sensor and define the time window used to query the database for new data.
877 Additional sensor and actuator variables can be added to the view and plotted together with the simulation outputs of the selected
878 soft sensor.
879

880 **S.1.3. Monitoring view**



881 **Figure S3.** Monitoring view. On the left, the four main sections of the STAMM Dashboard are shown. On the right, information is
882 displayed regarding the deployed soft sensor, the experiment ID to which the soft sensor is connected, and the drift detector
883 details. Additionally, a density plot is shown comparing the data used to train the model with the data from the connected
884 experiment for a specific model variable.
885

886 **S.1.4. Simulation assessment view**



887
888 **Figure S4.** Simulation assessment view. On the left, the four main sections of the STAMM Dashboard are displayed. On the
889 right, the predictions of all models used for the model divergence functionality are shown.

890 **S2 Metadata model**

Top-Level Section	Field	Subfield	Format	Description
model_identification	name	-	string	Name of the model.
	version	-	string	Model version identifier.
	ID	-	string	Internal or project-specific model code.
	UUID	-	string	Unique identifier for traceability.
	author	-	string	List of model authors.
	DOI	-	string	DOI link to related publication or source.
	creation_date	-	date	Date the metadata file was created.
	project	-	string	Reference to associated project configuration file.
	status	-	string	Deployment status (e.g., online/offline).
model_description	status_description	-	string	Additional explanation of the model status.
	learner	-	string	Name of the learning algorithm.
	model_type	-	string	Category of model (e.g., interpretable, black box, etc.).
	model_name	-	string	Descriptive title of the model.
	description	-	string	Textual explanation of model behavior.
	language	name	string	Programming language used.
		version	string	Programming language version.
	config_files	model_file	string	Path/filename of the serialized model.
		rest_api	string	REST endpoint used for predictions (optional)
packages	package	package	string	Name of a required software package.
	version	version	string	Version of the package.

	input_time_interval	time_interval → value	numeric	Numeric time interval between samples.
		time_interval → unit	string	Unit of the time interval (e.g., sec, minutes).
		aggregation → method	string	Method for resampling/aggregation.
		aggregation → description	string	Explanation of aggregation method.
		description	string	Description of the input interval definition.
model_architecture	layers, rules, etc.	-	key - value pairs	Defines the structure of the model.
training_information	number_of_instances	-	numeric	Number of training instances.
	hyperparameters	-	key - value pairs	Hyperparameters used during training.
	validation	-	string	Training validation procedure.
	experiments_ID	-	numeric/list	IDs of experiments used for training.
inputs	features	name	string	Name of the input feature.
		type	string	Sensor, actuator, computed variable.
		description	string	Explanation of the feature.
		units	string	Measurement units.
		lag	numeric	Time lag applied to the feature.
		feature_scaling	string	Scaling applied.
		expected_range → min	numeric	Minimum expected value.
		expected_range → max	numeric	Maximum expected value.
outputs	information	name	string	Name of the predicted variable.
		description	string	Explanation of the output variable.
		units	string	Measurement units.
		forecast_horizon	numeric	Prediction horizon.
		feature_scaling	string	Scaling applied to output.
		expected_range → min	numeric	Minimum expected output.
		expected_range → max	numeric	Maximum expected output.

Table S1. Structure of the model metadata showing sections, fields, and subfields

References

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., ... Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, 265–283.
- Acosta-Pavas, J. C., Corrales, D. C., Martínez-Menéndez, I., Villela, S. M. A., Bouhaouala-Zahar, B., Georgakilas, G. K., Mexis, K., Xenios, S., Dalamagas, T., & Kokosis, A. (2024). A Soft Sensing Approach for Efficient Monitoring of Nanobody-Based Scorpion Antivenom Production. *International Symposium on Distributed Computing and Artificial Intelligence*, 271–281.
- Acosta-Pavas, J. C., Robles-Rodriguez, C. E., Griol, D., Daboussi, F., Aceves-Lara, C. A., & Corrales, D. C. (2024). Soft sensors based on interpretable learners for industrial-scale fed-batch fermentation: Learning from simulations. *Computers & Chemical Engineering*, 187, 108736. <https://doi.org/10.1016/j.compchemeng.2024.108736>
- Ahmad, K., & Ansari, M. (2017). Hands-on influxdb. In *NoSQL* (pp. 341–354). Chapman and Hall/CRC.
- Amrit, C., & Narayanappa, A. K. (2025). An analysis of the challenges in the adoption of MLOps. *Journal of Innovation & Knowledge*, 10(1), 100637. <https://doi.org/10.1016/j.jik.2024.100637>
- Ancelin, M., Martins dos Santos, V. A. P., Morrissey, J. P., O'Donohue, M. J., Penttilä, M., & Philp, J. C. (2025). Addressing semantic ambiguity in biotechnology: Proposals from the European research infrastructure IBISBA. *New Biotechnology*, 88, 83–88. <https://doi.org/10.1016/j.nbt.2025.04.010>

917 Appl, C., Moser, A., Baganz, F., & Hass, V. C. (2021). Digital Twins for Bioprocess Control
 918 Strategy Development and Realisation. In C. Herwig, R. Pörtner, & J. Möller (Eds.),
 919 *Digital Twins: Applications to the Design and Optimization of Bioprocesses* (pp. 63–
 920 94). Springer International Publishing. https://doi.org/10.1007/10_2020_151
 921 Barker, M., Chue Hong, N. P., Katz, D. S., Lamprecht, A.-L., Martinez-Ortiz, C.,
 922 Psomopoulos, F., Harrow, J., Castro, L. J., Gruenpeter, M., Martinez, P. A., &
 923 Honeyman, T. (2022). Introducing the FAIR Principles for research software.
 924 *Scientific Data*, 9(1), 622. <https://doi.org/10.1038/s41597-022-01710-x>
 925 Basseville, M., & Nikiforov, I. V. (1993). *Detection of abrupt changes: Theory and application*
 926 (Vol. 104). Prentice hall Englewood Cliffs.
 927 Bergs, T., Gierlings, S., Auerbach, T., Klink, A., Schraknepper, D., & Augspurger, T. (2021).
 928 The Concept of Digital Twin and Digital Shadow in Manufacturing. *Procedia CIRP*,
 929 101, 81–84. <https://doi.org/10.1016/j.procir.2021.02.010>
 930 Bifet, A., & Gavaldà, R. (2007). Learning from Time-Changing Data with Adaptive
 931 Windowing. In *Proceedings of the 2007 SIAM International Conference on Data*
 932 *Mining (SDM)* (pp. 443–448). Society for Industrial and Applied Mathematics.
 933 <https://doi.org/10.1137/1.9781611972771.42>
 934 Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140.
 935 <https://doi.org/10.1007/BF00058655>
 936 Cao, L., Su, J., Conde, E., Siang, L. C., Cao, Y., & Gopaluni, B. (2025). A novel automated
 937 soft sensor design tool for industrial applications based on machine learning. *Control*
 938 *Engineering Practice*, 160, 106322.
 939 <https://doi.org/10.1016/j.conengprac.2025.106322>
 940 Child, A. W., Hinds, J., Sheneman, L., & Buerki, S. (2022). Centralized project-specific
 941 metadata platforms: Toolkit provides new perspectives on open data management

942 within multi-institution and multidisciplinary research projects. *BMC Research Notes*,
 943 15(1), 106. <https://doi.org/10.1186/s13104-022-05996-3>

944 Chollet, F. & others. (2015). *Keras*. <https://keras.io>

945 Cregan, V., Lee, W. T., & Clune, L. (2017). A soft sensor for the Bayer process. *Journal of*
 946 *Mathematics in Industry*, 7(1), 7. <https://doi.org/10.1186/s13362-017-0037-9>

947 Crowther, M., Metcalfe, B., Suarez, C., Corrales, D. C., Astudillo Lagos, J. A., & Koehorst, J.
 948 J. (2025). *LEAF: Laboratory equipment adapter framework*.

949 Dasu, T., Krishnan, S., Venkatasubramanian, S., & Yi, K. (2006). An information-theoretic
 950 approach to detecting changes in multi-dimensional data streams. *Proc. Symp. on*
 951 *the Interface of Statistics, Computing Science, and Applications*, 12(2), 1–22.

952 de Amorim, L. B. V., Cavalcanti, G. D. C., & Cruz, R. M. O. (2023). The choice of scaling
 953 technique matters for classification performance. *Applied Soft Computing*, 133,
 954 109924. <https://doi.org/10.1016/j.asoc.2022.109924>

955 Dietterich, T. G. (2000). Ensemble Methods in Machine Learning. *Multiple Classifier*
 956 *Systems*, 1–15. https://doi.org/10.1007/3-540-45014-9_1

957 Fernandes, M. de A., Gildin, E., & Sampaio, M. A. (2025). Machine Learning-Based Virtual
 958 Sensor for Bottom-Hole Pressure Estimation in Petroleum Wells. *Eng*, 6(11), 318.
 959 <https://doi.org/10.3390/eng6110318>

960 Fortuna, L., Graziani, S., Rizzo, A., & Xibilia, M. G. (Eds.). (2007). Soft Sensors in Industrial
 961 Applications. In *Soft Sensors for Monitoring and Control of Industrial Processes* (pp.
 962 1–13). Springer. https://doi.org/10.1007/978-1-84628-480-9_1

963 Gábor, A., & Banga, J. R. (2015). Robust and efficient parameter estimation in dynamic
 964 models of biological systems. *BMC Systems Biology*, 9(1), 74.
 965 <https://doi.org/10.1186/s12918-015-0219-2>

- 966 Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on
967 concept drift adaptation. *ACM Computing Surveys (CSUR)*, 46(4), 1–37.
- 968 Gargalo, C. L., de las Heras, S. C., Jones, M. N., Udugama, I., Mansouri, S. S., Krühne, U.,
969 & Gernaey, K. V. (2021). Towards the Development of Digital Twins for the Bio-
970 manufacturing Industry. In C. Herwig, R. Pörtner, & J. Möller (Eds.), *Digital Twins:
971 Tools and Concepts for Smart Biomanufacturing* (pp. 1–34). Springer International
972 Publishing. https://doi.org/10.1007/10_2020_142
- 973 Goldrick, S., Duran-Villalobos, C. A., Jankauskas, K., Lovett, D., Farid, S. S., & Lennox, B.
974 (2019). Modern day monitoring and control challenges outlined on an industrial-scale
975 benchmark fermentation process. *Computers & Chemical Engineering*, 130, 106471.
976 <https://doi.org/10.1016/j.compchemeng.2019.05.037>
- 977 Goldrick, S., Ştefan, A., Lovett, D., Montague, G., & Lennox, B. (2015). The development of
978 an industrial-scale fed-batch fermentation simulation. *Journal of Biotechnology*, 193,
979 70–82. <https://doi.org/10.1016/j.jbiotec.2014.10.029>
- 980 Harenslak, B. P., & De Ruiter, J. (2021). *Data pipelines with apache airflow*. Simon and
981 Schuster.
- 982 Heil, B. J., Hoffman, M. M., Markowetz, F., Lee, S.-I., Greene, C. S., & Hicks, S. C. (2021).
983 Reproducibility standards for machine learning in the life sciences. *Nature Methods*,
984 18(10), 1132–1135. <https://doi.org/10.1038/s41592-021-01256-7>
- 985 Hjelle, S., Mikalef, P., Altwaijry, N., & Parida, V. (2024). Organizational decision making and
986 analytics: An experimental study on dashboard visualizations. *Information &
987 Management*, 61(6), 104011. <https://doi.org/10.1016/j.im.2024.104011>
- 988 Jones, D. T. (2019). Setting the standards for machine learning in biology. *Nature Reviews
989 Molecular Cell Biology*, 20(11), 659–660. <https://doi.org/10.1038/s41580-019-0176-5>

- 990 Kadlec, P., Gabrys, B., & Strandt, S. (2009). Data-driven Soft Sensors in the process
991 industry. *Computers & Chemical Engineering*, 33(4), 795–814.
992 <https://doi.org/10.1016/j.compchemeng.2008.12.012>
- 993 Kant, R., & Gurung, H. (Eds.). (2023). *Industry 4.0: Concepts, Processes and Systems*. CRC
994 Press. <https://doi.org/10.1201/9781003246466>
- 995 Kim, S., Lee, M., Kim, D., Yoon, S., & Lee, D. (2025). AutoML-Driven Soft Sensors for
996 Real-Time Monitoring of Amino Acids in Mammalian Perfusion Cultures.
997 *Biotechnology and Bioengineering*, 122(11), 3123–3138.
998 <https://doi.org/10.1002/bit.70036>
- 999 Knibbe, W. J., Afman, L., Boersma, S., Bogaardt, M.-J., Evers, J., Van Evert, F., Van der
1000 Heide, J., Hoving, I., Van Mourik, S., & De Ridder, D. (2022). Digital twins in the
1001 green life sciences. *NJAS: Impact in Agricultural and Life Sciences*, 94(1), 249–279.
- 1002 Krishna, V. V. S. V., Pappa, N., & Rani, S. P. J. V. (2021). Deep Learning based Soft Sensor
1003 for Bioprocess Application. *2021 IEEE Second International Conference on Control,*
1004 *Measurement and Instrumentation (CMI)*, 155–159.
1005 <https://doi.org/10.1109/CMI50323.2021.9362948>
- 1006 Kritzinger, W., Karner, M., Traar, G., Henjes, J., & Sihn, W. (2018). Digital Twin in
1007 manufacturing: A categorical literature review and classification. *IFAC-PapersOnLine*,
1008 51(11), 1016–1022. <https://doi.org/10.1016/j.ifacol.2018.08.474>
- 1009 Kuncheva, L. I., & Whitaker, C. J. (2003). Measures of Diversity in Classifier Ensembles and
1010 Their Relationship with the Ensemble Accuracy. *Machine Learning*, 51(2), 181–207.
1011 <https://doi.org/10.1023/A:1022859003006>
- 1012 Li, J., Chen, Z., Zhong, X., Li, X., Xia, X., & Liu, B. (2025). Design of Soft-Sensing Model for
1013 Alumina Concentration Based on Improved Grey Wolf Optimization Algorithm and
1014 Deep Belief Network. *Processes*, 13(3), 606. <https://doi.org/10.3390/pr13030606>

- 1015 Lipton, Z. C. (2018). The mythos of model interpretability. *Commun. ACM*, 61(10), 36–43.
 1016 <https://doi.org/10.1145/3233231>
- 1017 Liu, Y., Ling, Z., Huo, B., Wang, B., Chen, T., & Mouine, E. (2020). Building A Platform for
 1018 Machine Learning Operations from Open Source Frameworks. *IFAC-PapersOnLine*,
 1019 53(5), 704–709. <https://doi.org/10.1016/j.ifacol.2021.04.161>
- 1020 Margaria, T., & Schieweck, A. (2019). The Digital Thread in Industry 4.0. In W. Ahrendt & S.
 1021 L. Tapia Tarifa (Eds.), *Integrated Formal Methods* (pp. 3–24). Springer International
 1022 Publishing. https://doi.org/10.1007/978-3-030-34968-4_1
- 1023 Mehdiyev, N., Majlatow, M., & Fettke, P. (2025). Interpretable and explainable machine
 1024 learning methods for predictive process monitoring: A systematic literature review.
 1025 *Artificial Intelligence Review*, 58(12), 378. [https://doi.org/10.1007/s10462-025-11399-](https://doi.org/10.1007/s10462-025-11399-0)
 1026 0
- 1027 Meier, M. (2023). *Mastering Tableau 2023: Implement advanced business intelligence*
 1028 *techniques, analytics, and machine learning models with Tableau*. Packt Publishing
 1029 Ltd.
- 1030 Metcalfe, B., Acosta-Pavas, J. C., Robles-Rodriguez, C. E., Georgakilas, G. K., Dalamagas,
 1031 T., Aceves-Lara, C. A., Daboussi, F., Koehorst, J. J., & Corrales, D. C. (2025).
 1032 Towards a machine learning operations (MLOps) soft sensor for real-time predictions
 1033 in industrial-scale fed-batch fermentation. *Computers & Chemical Engineering*, 194,
 1034 108991. <https://doi.org/10.1016/j.compchemeng.2024.108991>
- 1035 Mishra, R., Singh, R., & Majumder, C. B. (2025). Forecasting biochemical oxygen demand
 1036 (BOD) in River Ganga: A case study employing supervised machine learning and
 1037 ANN techniques. *Sustainable Water Resources Management*, 11(1), 9.
 1038 <https://doi.org/10.1007/s40899-024-01188-y>

- 1039 Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E.,
 1040 Raji, I. D., & Gebru, T. (2019). Model Cards for Model Reporting. *Proceedings of the*
 1041 *Conference on Fairness, Accountability, and Transparency*, 220–229.
 1042 <https://doi.org/10.1145/3287560.3287596>
- 1043 Mojto, M., Ľubušký, K., Fikar, M., & Paulen, R. (2021). Data-based design of inferential
 1044 sensors for petrochemical industry. *Computers & Chemical Engineering*, 153,
 1045 107437. <https://doi.org/10.1016/j.compchemeng.2021.107437>
- 1046 Nasir, A. H. M., Surin, E. S. M., & Ahmad, M. N. (2024). Towards Unified Information
 1047 Dashboard Design—Common Principles, Practices, and Challenges. *Procedia*
 1048 *Computer Science*, 234, 1586–1592. <https://doi.org/10.1016/j.procs.2024.03.161>
- 1049 Opitz, D., & Maclin, R. (1999). Popular ensemble methods: An empirical study. *J. Artif. Int.*
 1050 *Res.*, 11(1), 169–198.
- 1051 Pattnaik, B. S., Pattanayak, A. S., Udgata, S. K., & Panda, A. K. (2021). Machine learning
 1052 based soft sensor model for BOD estimation using intelligence at edge. *Complex &*
 1053 *Intelligent Systems*, 7(2), 961–976. <https://doi.org/10.1007/s40747-020-00259-9>
- 1054 Pears, R., Sakthithasan, S., & Koh, Y. S. (2014). Detecting concept change in dynamic data
 1055 streams: A sequential approach based on reservoir sampling. *Machine Learning*, 97,
 1056 259–293.
- 1057 Pinheiro, J. M. H., Oliveira, S. V. B. de, Silva, T. H. S., Saraiva, P. A. R., Souza, E. F. de,
 1058 Godoy, R. V., Ambrosio, L. A., & Becker, M. (2025). *The Impact of Feature Scaling In*
 1059 *Machine Learning: Effects on Regression and Classification Tasks* (No.
 1060 arXiv:2506.08274). arXiv. <https://doi.org/10.48550/arXiv.2506.08274>
- 1061 Powell, B. (2018). *Mastering Microsoft Power BI: Expert techniques for effective data*
 1062 *analytics and business intelligence*. Packt Publishing Ltd.

1063 Qahtan, A. A., Alharbi, B., Wang, S., & Zhang, X. (2015). A PCA-Based Change Detection
 1064 Framework for Multidimensional Data Streams: Change Detection in
 1065 Multidimensional Data Streams. *Proceedings of the 21th ACM SIGKDD International*
 1066 *Conference on Knowledge Discovery and Data Mining*, 935–944.
 1067 <https://doi.org/10.1145/2783258.2783359>
 1068 Rahman, Md. S., Ghosh, T., Aurna, N. F., Kaiser, M. S., Anannya, M., & Hosen, A. S. M. S.
 1069 (2023). Machine learning and internet of things in industry 4.0: A review.
 1070 *Measurement: Sensors*, 28, 100822. <https://doi.org/10.1016/j.measen.2023.100822>
 1071 Rosenthal, K., Lindley, M., Turner, M., Ratcliffe, E., & Hunsicker, E. (2024). Current data
 1072 processing methods and reporting standards for untargeted analysis of volatile
 1073 organic compounds using direct mass spectrometry: A systematic review.
 1074 *Metabolomics*, 20(2), 42. <https://doi.org/10.1007/s11306-024-02104-3>
 1075 Rudin, C. (2019). Stop explaining black box machine learning models for high stakes
 1076 decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5),
 1077 206–215. <https://doi.org/10.1038/s42256-019-0048-x>
 1078 Santos, A. de M., & Sant'Anna, Â. M. O. (2024). Industry 4.0 technologies for sustainability
 1079 within small and medium enterprises: A systematic literature review and future
 1080 directions. *Journal of Cleaner Production*, 467, 143023.
 1081 <https://doi.org/10.1016/j.jclepro.2024.143023>
 1082 Schroeder, A., Mayer, C., & Ward, A. M. (2022). *The book of Dash: Build dashboards with*
 1083 *Python and Plotly*. No Starch Press.
 1084 Schweidtmann, A. M., Zhang, D., & von Stosch, M. (2024). A review and perspective on
 1085 hybrid modeling methodologies. *Digital Chemical Engineering*, 10, 100136.
 1086 <https://doi.org/10.1016/j.dche.2023.100136>

- 1087 Shyu, H.-Y., Castro, C. J., Bair, R. A., Lu, Q., & Yeh, D. H. (2023). Development of a Soft
1088 Sensor Using Machine Learning Algorithms for Predicting the Water Quality of an
1089 Onsite Wastewater Treatment System. *ACS Environmental Au*, 3(5), 308–318.
1090 <https://doi.org/10.1021/acsenvironau.2c00072>
- 1091 Soori, M., Jough, F. K. G., Dastres, R., & Arezoo, B. (2026). AI-based decision support
1092 systems in Industry 4.0, a review. *Journal of Economy and Technology*, 4, 206–225.
1093 <https://doi.org/10.1016/j.ject.2024.08.005>
- 1094 Taplin, R., & Hunt, C. (2019). The Population Accuracy Index: A New Measure of Population
1095 Stability for Model Monitoring. *Risks*, 7(2), Article 2.
1096 <https://doi.org/10.3390/risks7020053>
- 1097 Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Miyazaki,
1098 M., & Heidmann, L. (2020). *Introducing MLOps*. O'Reilly Media.
- 1099 Uma, A. N., Fornaciari, T., Hovy, D., Paun, S., Plank, B., & Poesio, M. (2022). Learning from
1100 Disagreement: A Survey. *J. Artif. Int. Res.*, 72, 1385–1470.
1101 <https://doi.org/10.1613/jair.1.12752>
- 1102 von Stosch, M., Oliveira, R., Peres, J., & Feyer de Azevedo, S. (2014). Hybrid semi-
1103 parametric modeling in process systems engineering: Past, present and future.
1104 *Computers & Chemical Engineering*, 60, 86–101.
1105 <https://doi.org/10.1016/j.compchemeng.2013.08.008>
- 1106 Walsh, I., Fishman, D., Garcia-Gasulla, D., Titma, T., Pollastri, G., Harrow, J., Psomopoulos,
1107 F. E., & Tosatto, S. C. E. (2021). DOME: Recommendations for supervised machine
1108 learning validation in biology. *Nature Methods*, 18(10), 1122–1127.
1109 <https://doi.org/10.1038/s41592-021-01205-4>

1110 Walsh, I., Pollastri, G., & Tosatto, S. C. E. (2016). Correct machine learning on protein
 1111 sequences: A peer-reviewing perspective. *Briefings in Bioinformatics*, 17(5), 831–
 1112 840. <https://doi.org/10.1093/bib/bbv082>
 1113 Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A.,
 1114 Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., Bouwman, J.,
 1115 Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T.,
 1116 Finkers, R., ... Mons, B. (2016). The FAIR Guiding Principles for scientific data
 1117 management and stewardship. *Scientific Data*, 3(1), 160018.
 1118 <https://doi.org/10.1038/sdata.2016.18>
 1119 Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining.
 1120 *Proceedings of the 4th International Conference on the Practical Applications of*
 1121 *Knowledge Discovery and Data Mining*, 1, 29–39.
 1122 Wolstencroft, K., Krebs, O., Snoep, J. L., Stanford, N. J., Bacall, F., Golebiewski, M.,
 1123 Kuzyakiv, R., Nguyen, Q., Owen, S., Soiland-Reyes, S., Straszewski, J., van Niekerk,
 1124 D. D., Williams, A. R., Malmström, L., Rinn, B., Müller, W., & Goble, C. (2017).
 1125 FAIRDOMHub: A repository and collaboration environment for sharing systems
 1126 biology research. *Nucleic Acids Research*, 45(D1), D404–D407.
 1127 <https://doi.org/10.1093/nar/gkw1032>
 1128 Wolstencroft, K., Owen, S., Du Preez, F., Krebs, O., Mueller, W., Goble, C., & Snoep, J. L.
 1129 (2011). The SEEK: a platform for sharing data and models in systems biology. In
 1130 *Methods in enzymology* (Vol. 500, pp. 629–655). Elsevier.
 1131 Wolstencroft, K., Owen, S., Krebs, O., Nguyen, Q., Stanford, N. J., Golebiewski, M.,
 1132 Weidemann, A., Bittkowski, M., An, L., Shockley, D., Snoep, J. L., Mueller, W., &
 1133 Goble, C. (2015). SEEK: A systems biology data and model management platform.
 1134 *BMC Systems Biology*, 9(1), 33. <https://doi.org/10.1186/s12918-015-0174-y>

1135 Wright, L., & Davidson, S. (2020). How to tell the difference between a model and a digital
 1136 twin. *Advanced Modeling and Simulation in Engineering Sciences*, 7(1), 13.
 1137 <https://doi.org/10.1186/s40323-020-00147-4>

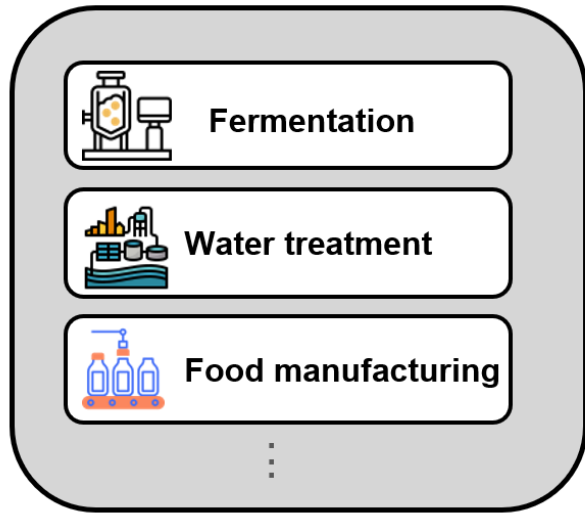
1138 Yan, W., Tang, D., & Lin, Y. (2017). A Data-Driven Soft Sensor Modeling Method Based on
 1139 Deep Learning and its Application. *IEEE Transactions on Industrial Electronics*,
 1140 64(5), 4237–4245. *IEEE Transactions on Industrial Electronics*.
 1141 <https://doi.org/10.1109/TIE.2016.2622668>

1142 Zhou, Z.-H. (2012). *Ensemble methods: Foundations and algorithms*. CRC press.

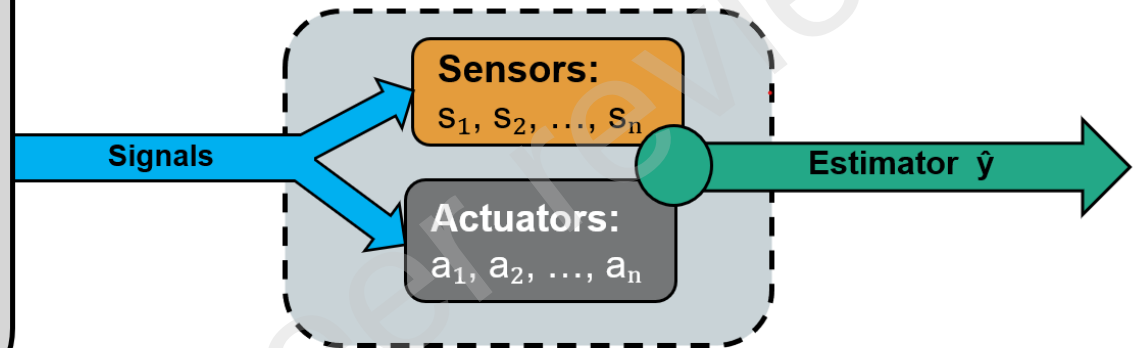
1143 Zhu, X., Rehman, K. U., Wang, B., Shahzad, M., Zhu, X., Rehman, K. U., Wang, B., &
 1144 Shahzad, M. (2020). Modern Soft-Sensing Modeling Methods for Fermentation
 1145 Processes. *Sensors*, 20(6). <https://doi.org/10.3390/s20061771>

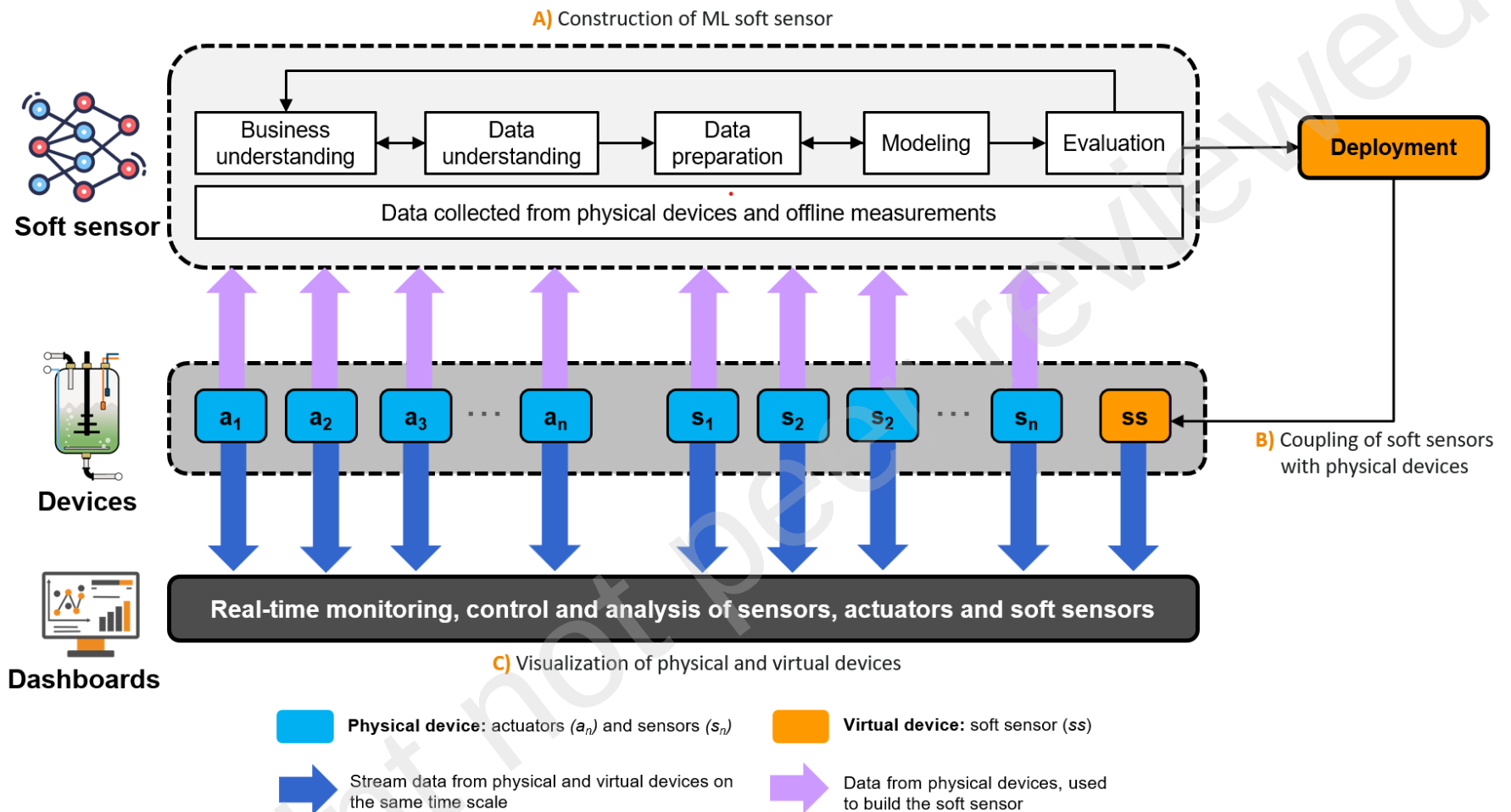
1146

Industrial process

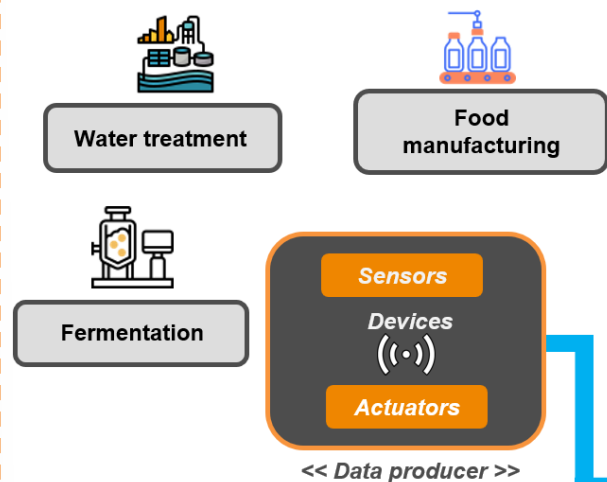


Soft sensor

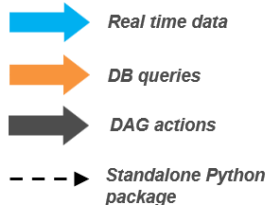




Physical world: industrial process

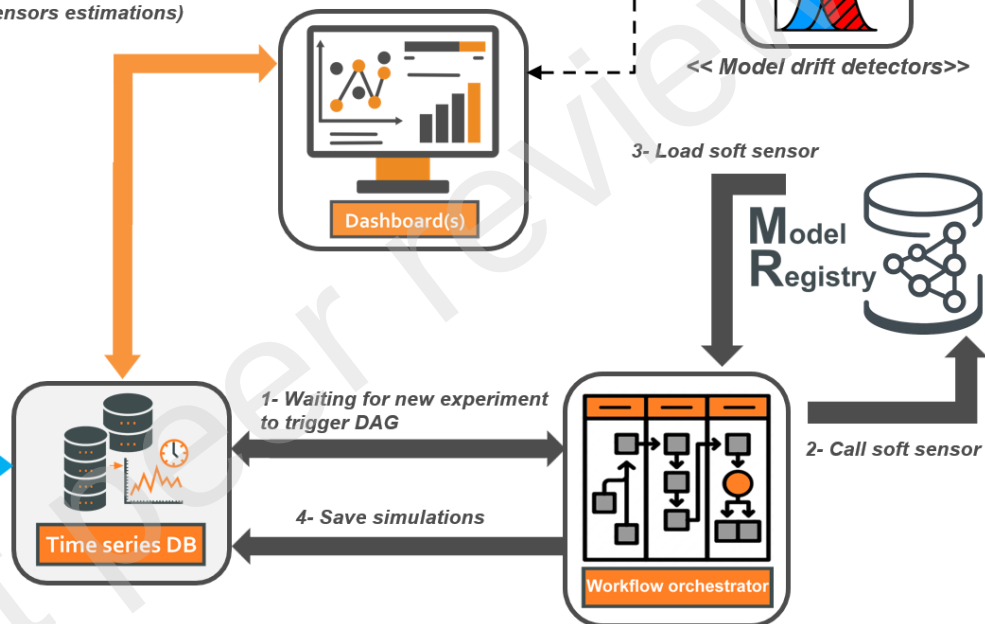


Legend:



Virtual world: STAMM infrastructure

Waiting for new data to show (e.g., sensors, actuators, soft sensors estimations)



Check DB connection

- Verify InfluxDB service availability
- Validate required environment variables (URL, token, org)
- Perform client ping() to ensure backend responsiveness
- Halt or retry DAG execution if connectivity fails

Detect new data

- Identify latest timestamp per tag-group
- Assess freshness/novelty
- Pivot observed_property → wide snapshot
- Push snapshots to XCom

Run ML Models

- Discover models via API
- Retrieve metadata (YAML → JSON)
- Execute POST /predict
- Parse results + version tracking
- Push predictions (timestamp-aligned)

Store predictions

- Write to stamm_predictions bucket
- Tags: model_id, version, observed_property
- Time = snapshot_time (from stamm_raw)

Data source

Experiment Data Browser:

Batch 5 – last data 2 min. ago]



Project Name:

<< Project description >>

Statistical summary of the variables from experiment: Batch 5

Type	Name	Unit	Mean	Max	Min
Sensor					
Actuator					

Recent Measurements

Value:

Unit:

Days



Experiment ID	Start time	End time	Batch size
Batch 5			
Batch 7			

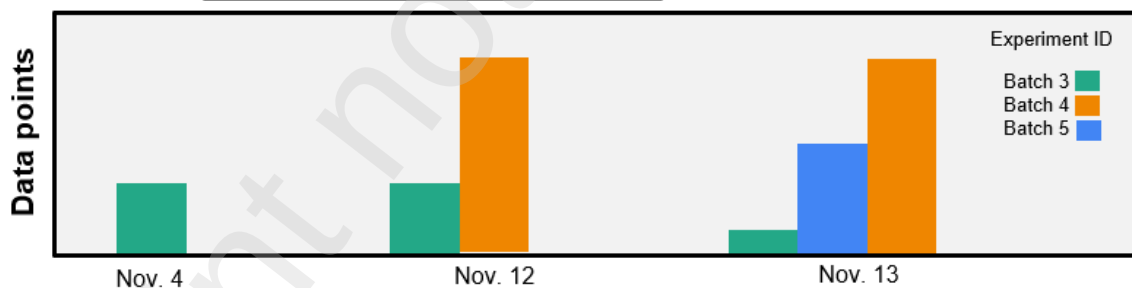
Data Point Distribution

Value:

11/01/2025 → 11/13/2025

Variable:

Agitator



Soft sensors

Model name: LSTM [python version] ▼

Run model

Interval update (seconds): 5



Model metadata: LSTM [python version] ▼

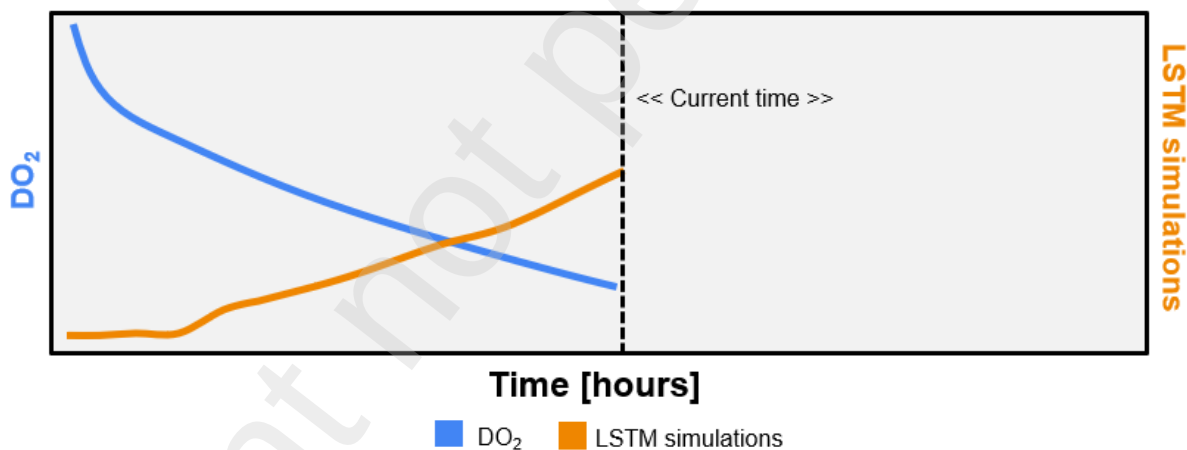
```
name: "LSTM"  
version: "V.1.0"  
ID: "0009_[Python]_penicillin_LSTM"  
author: "Suarez, C., Astudillo A., Metcalfe B., Koehorst J.J., Castillo E. & Corrales, D. C"  
status: "online"
```

Variable: Sensor ▼

Name: DO₂ ▼

Add

Experiment ID: Batch 5



Monitoring

Deployed model:

LSTM [python version]



Drift detector :

ADWIN – Adaptive WINdowing



Experiment ID: Batch 5

Time range selection (data points):



Metric parameters:

Variable:

DO₂



Drift detector parameters:

θ

ϵ

Explanation:

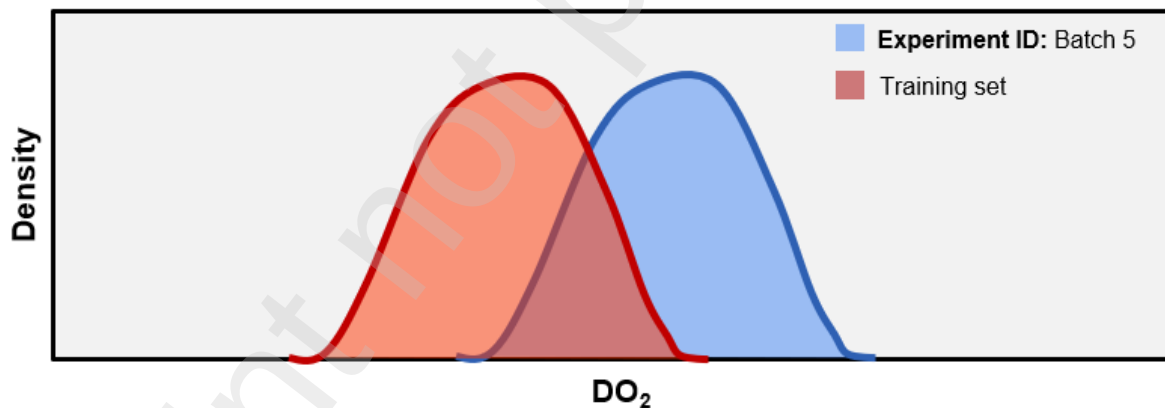
<< Drift detector description >>

Drift result:

<< Drift detector score and metadata >>

Run

Density plot of training set and Experiment ID: 5 – Variable: DO₂



Simulation assessment

Deployed model:

LSTM [python version] ▼

Experiment ID: Batch 5

Time range selection (data points):



Variable:

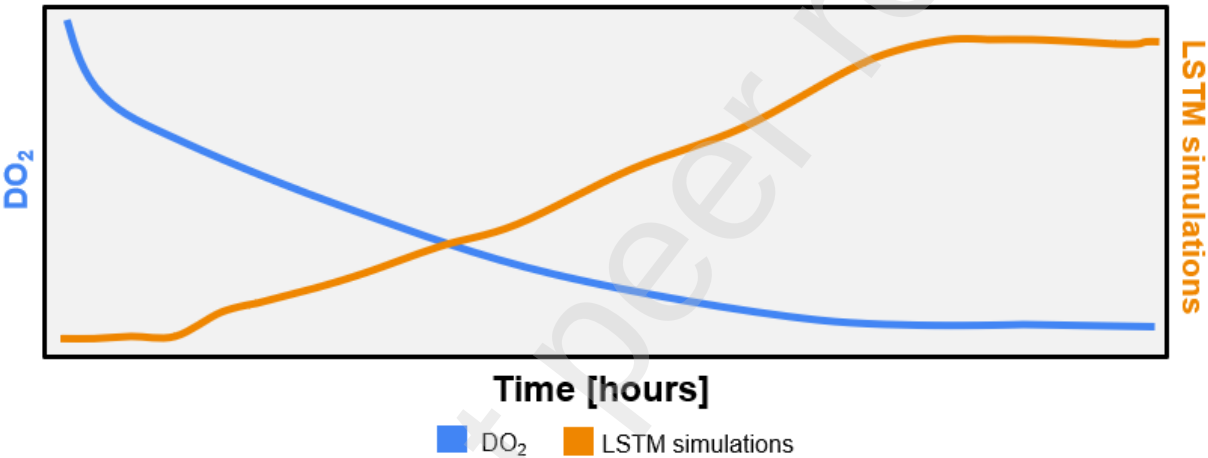
Sensor ▼

Name:

DO₂ ▼

Add

Experiment ID: Batch 5



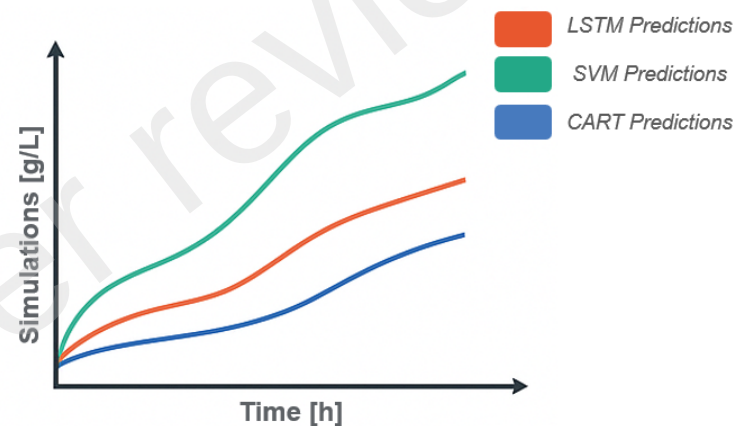
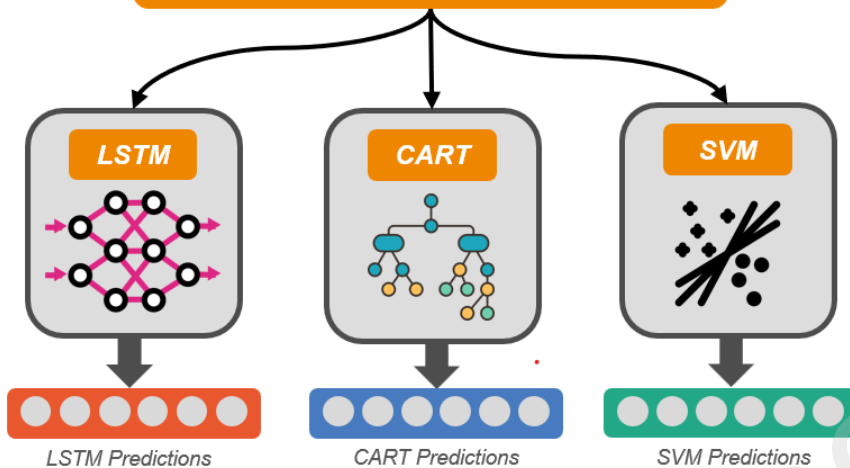
Variable:

Select a column ▼

Enter a value:

Time	DO2	LSTM simulations	Report data point

Input X samples



Model divergence metrics

$$CV = \frac{\sigma}{\mu} \quad \dots \quad \rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y}$$

High model divergence?



Soft sensor monitoring and maintenance
framework for Machine learning Models.

Welcome, admin

1 Data source

2 Soft sensors

3 Monitoring

4 Simulation assessment

About us

Help

Admin Panel

Logout

Data source

Please choose an Experiment ID to load the available models:

batch_31 - last data 17 days ago

Project name: IndPenSim

Project description:

The dataset used in this work corresponds to a series of experiments that were subsequently utilized to produce a simulation of industrial-scale penicillin fermentation processes (Paul and Thomas, 1996) which describes all the component balances relating to the process variables (Goldrick et al., 2015, 2019). The dataset generated from the industrial-scale penicillin fermentation (IndPenSim) includes 100 batches with each dataset comprising 2238 variables of which 39 variables correspond to process variables (manual and automatic control and online and offline measurements) and the remaining 2199 correspond to Raman spectra. Per batch sensors recorded data every 12 min with the average batch length being approximately 230 h. Out of the 100 batches, the first 90 batches were operated under "normal" conditions using three different control strategies: (I) controlled by a recipe driven approach; (II) controlled by an operator and (III) controlled by an Advanced Process Control (APC). The last 10 batches contain faults, resulting in process deviations (Goldrick et al., 2019).

Statistical summary

Experiment Duration: 3 hours

Statistical summary of the variables from the chosen experiment: batch_31

Type	Name	Unit	Mean	Max	Min
Sensor	CO2_percent_in_off_gas	N/A	1.1351	2.1666	0.0816
Actuator	agitator	N/A	100	100	100
Sensor	dissolved_oxygen_concentration	N/A	13.3113	15.904	9.6317
Sensor	oxygen_in_percent_in_off_gas	N/A	0.1905	0.2064	0.1793
Sensor	pH	N/A	6.5011	6.6563	6.4018
Sensor	penicillin_concentration	N/A	4.7056	14.187	0.0009
Actuator	sugar_feed_rate	N/A	64.0761	150	8
Sensor	temperature	N/A	298.1062	299.45	297.66
Computed Variable	vessel_volume	N/A	66548.3304	84239	57316

Time Unit for 'Recent Measurements'

5



Soft sensor monitoring and maintenance framework for Machine learning Models.

Welcome, admin

- 1 Data source
- 2 Soft sensors**
- 3 Monitoring
- 4 Simulation assessment

About us

Help

Admin Panel

Logout

Soft sensors

Model Name:

LSTM without lag features [Python version]



Show prediction

Interval update (seconds): 5

Model name: LSTM without lag features [Python version]



Variable:

sensor



Name:

dissolved_oxygen_concentration



Add Variable

Variable monitoring

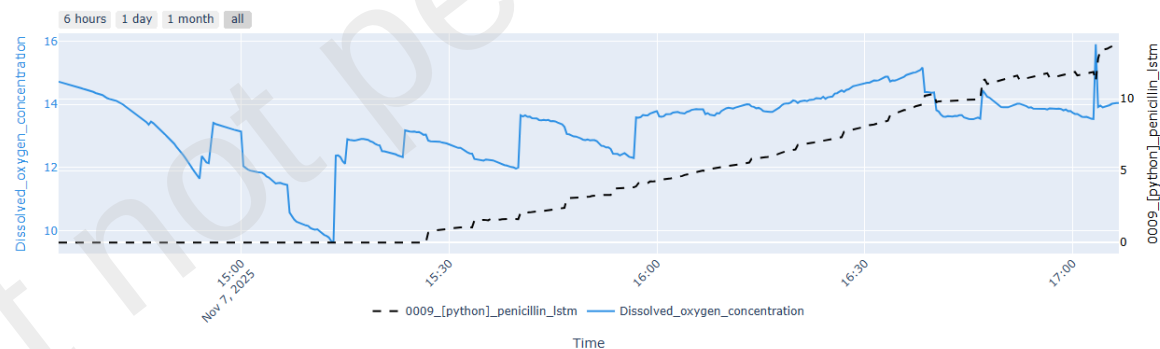
Action

dissolved_oxygen_concentration

(Remove)

Selected Experiment ID: batch_31

Performance Penicillin





Soft sensor monitoring and maintenance framework for Machine learning Models.

Welcome, admin

- 1 Data source
- 2 Soft sensors
- 3 Monitoring
- 4 Simulation assessment

About us

Help

Admin Panel

Logout

Monitoring soft sensor:

LSTM without lag features [Python version]

Experiment ID:

batch_31

Drift detector:

ADWIN - ADaptive WINdowing

Time range selection:

0 460
Last 460 elements - Selected 460
From: 2025-11-07 14:33:39 To: 2025-11-07 17:06:39

Metric Parameters

Variable

dissolved_oxygen_concentration

delta

0.001

Explanation

- ADWIN detected a real change in the data.
- The drift was detected at position 447 in the stream.
- Used parameters:
 - delta: This is a sensitivity parameter.
 - mode: It was running in online mode.

Drift Result

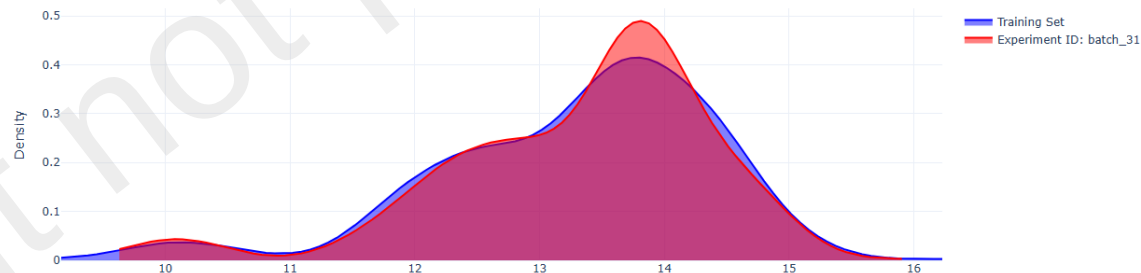
Drift Detected ✓

- last_index: 447
- drift: True
- details:
 - delta: 0.001
 - mode: online

Run

Monitored inputs

Density of training set and Experiment ID: batch_31 - Variable: dissolved_oxygen_concentration





Soft sensor monitoring and maintenance framework for Machine learning Models.

Welcome, admin

- 1 Data source
- 2 Soft sensors
- 3 Monitoring
- 4 Simulation assessment

About us

Help

Admin Panel

Logout

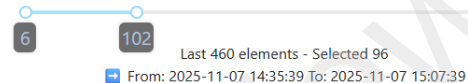
Deployed model:
LSTM without lag features [Python version]

Selected experiment:
batch_31

Monitoring soft sensor:

- × Hist Gradient Boosting Regressor [Python version]
- × CART (Classification And Regression Tree) [Python version]

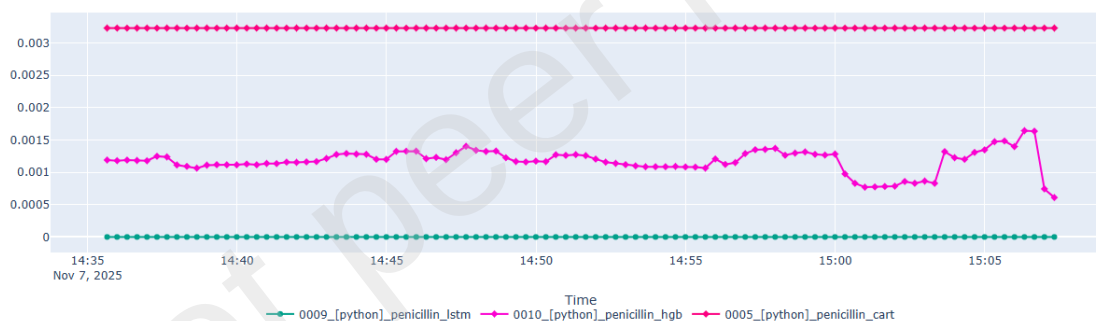
Time range selection:



Add

Reset

Select performance estimator



Show Detailed Metrics

Detailed Prediction and Metrics Table

Performance estimator:

CV - Coefficient of Variation

Model	0009_[Python]_penicillin_LSTM	0010_[Python]_penicillin_HGB	0005_[python]_penicillin_CART

STAMM: **S**oft sensor moni**T**oring and m**A**intenance framework for **M**achine learning **M**odels

Carlos Suarez¹, Alexander Astudillo², Brett Metcalfe^{3,4}, Matthew Crowther⁵, Jasper J Koehorst^{3,6}, Esteban Castillo⁷, Ariane Bize⁸ and David Camilo Corrales^{9*}

¹ Departamento de Telemática, Universidad del Cauca, Popayán 190002, Colombia

² INRAE, UMS (1337) TWB, 135 Avenue de Rangueil, 31077 Toulouse, France

³ Laboratory of Systems and Synthetic Biology, Wageningen University & Research, Wageningen, The Netherlands

⁴ Department of Earth Sciences, Faculty of Science, Vrije Universiteit Amsterdam, Amsterdam, The Netherlands

⁵ School of Computing Science, Newcastle University (NCL), Newcastle Upon Tyne, United Kingdom

⁶ UNLOCK, Wageningen University & Research and Delft University of Technology, The Netherlands.

⁷ Tecnológico de Monterrey, School of Engineering and Sciences, México

⁸ Université Paris-Saclay, INRAE, PROSE, 92160 Antony, France.

⁹ TBI, Université de Toulouse, CNRS, INRAE, INSA, Toulouse, France

*Corresponding author: David-Camilo.Corrales-Munoz@inrae.fr

ORCID:

0009-0009-1867-7304	Carlos Suarez
0000-0003-0475-4275	Jairo Alexander Astudillo Lagos
0000-0002-5873-9815	Brett Metcalfe
0000-0002-0497-2769	Matthew Crowther
0000-0001-8172-8981	Jasper J. Koehorst
0000-0001-7781-3999	Esteban Castillo
0000-0003-4023-8665	Ariane Bize
0000-0003-4717-3040	David Camilo Corrales

CRediT authorship contribution statement

Conceptualization: D.C.C & B.M.; **Data Curation:** A.A.; **Formal Analysis:** D.C.C.; **Funding Acquisition:** D.C.C., J.J.K., & A.B.; **Investigation:** All authors; **Methodology:** All authors; **Project Administration:** D.C.C.; **Resources:** D.C.C., J.J.K. & A.B.; **Software:** C.S., A.A., M.C., & D.C.C.; **Supervision:** D.C.C. & B.M.; **Validation:** A.B., E.C., & J.J.K. **Visualization:** C.S., E.C., B.M. & D.C.C.; **Writing - Original Draft:** D.C.C.; **Writing - Review & Editing:** B.M., J.J.K., A.B & D.C.C.

Acknowledgements

Brett Metcalfe was affiliated with the Department of Earth Sciences, Faculty of Science, Vrije Universiteit Amsterdam (The Netherlands) and Amsterdam University College (The Netherlands) at the same time as when a portion of this research was conducted. Carlos Suarez and David Camilo Corrales acknowledge the support of the Ministry of Science, Technology, and Innovation (MinCiencias, Colombia).

Funding

The authors disclose receipt of the following financial support for the research, authorship, and publication of this article: European Union's Horizon 2020 research and innovation programme projects 'RI services to promote deep digitalization of Industrial Biotechnology - towards smart biomanufacturing' (BIOINDUSTRY 4.0, grant agreement n° 101094287 [<https://doi.org/10.3030/101094287>]). Carlos Suarez was funded through Call 823 - High-Level Training of Human Capital for the Regions of Cauca (MinCiencias). Jasper J. Koehorst acknowledges the Dutch Research Council (NWO), and Wageningen University & Research for their financial contribution to the UNLOCK initiative (NWO: 184.035.007).